

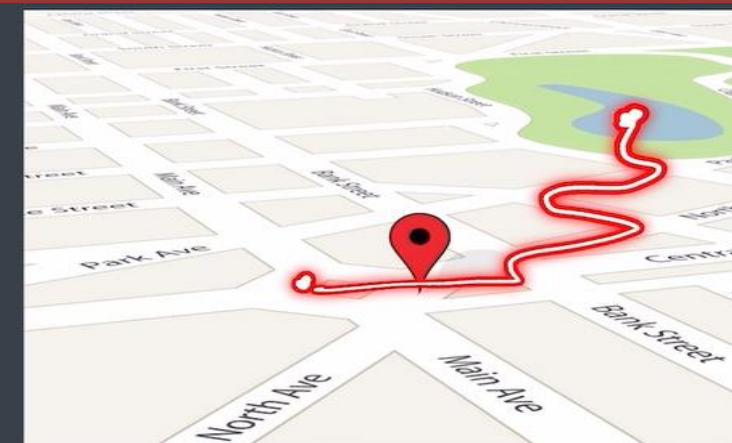
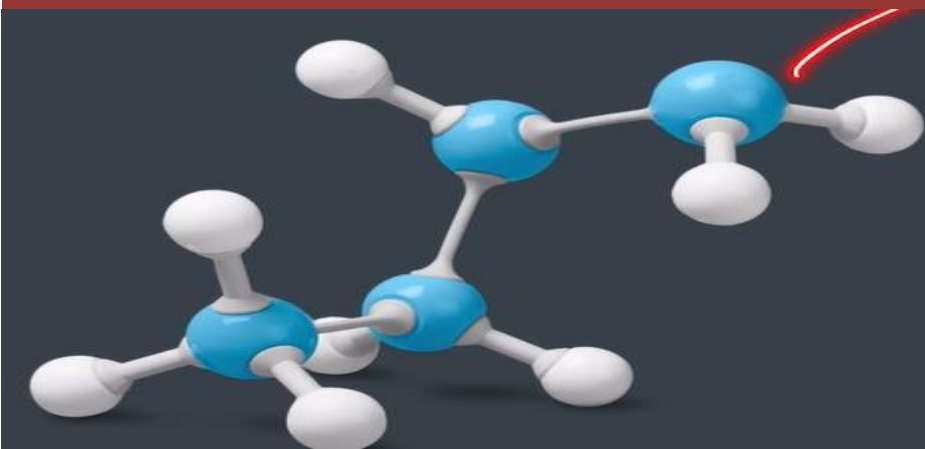


CSC205 ARTIFICIAL INTELLIGENCE

Lecture # : 07

Topic: Introduction to Search Algorithms

Arifa Awan



Learning Outcomes



By the end of this lecture, students will be able to:



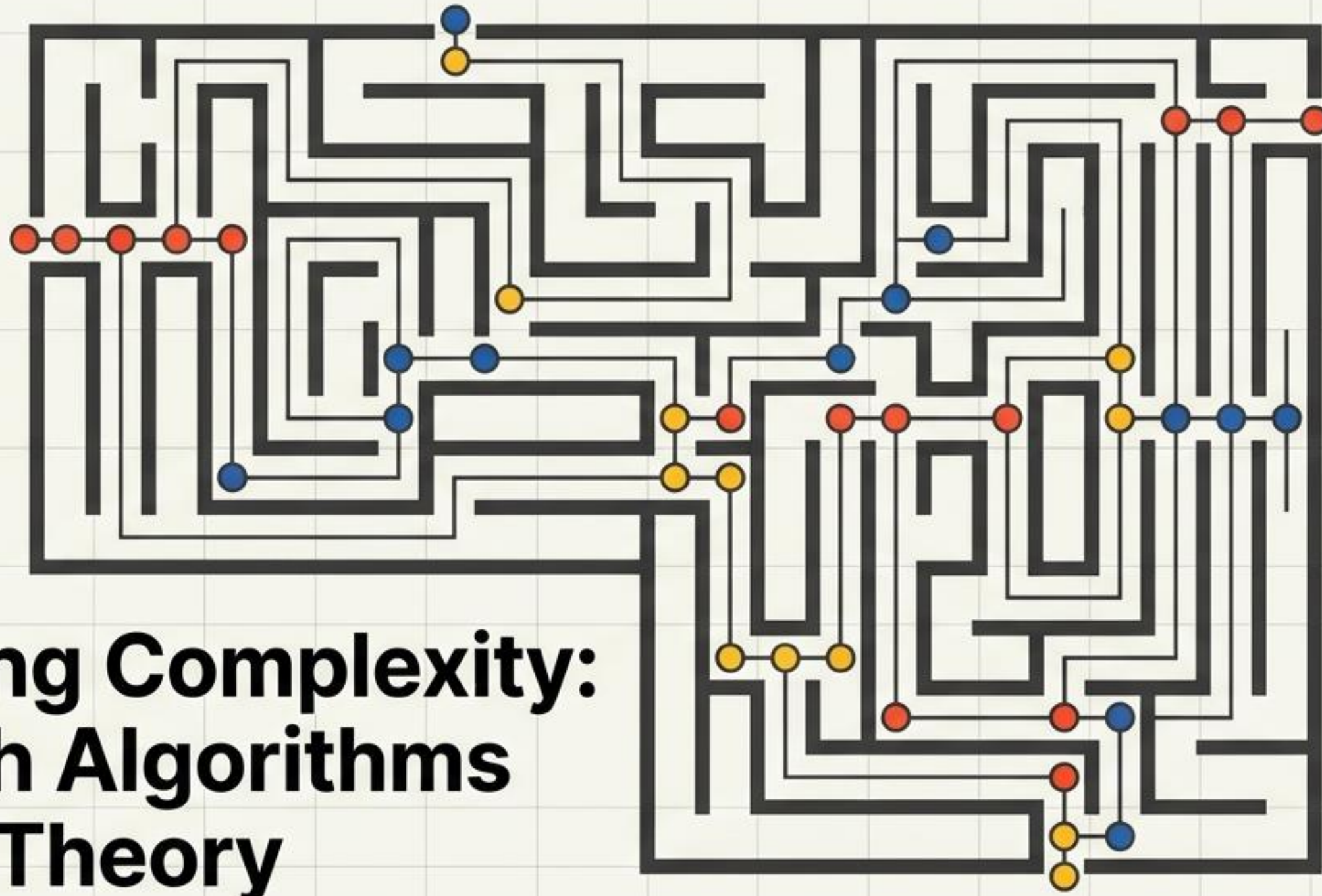
Explain the concept and importance of search algorithms in AI problem-solving.



Identify the components and different types of search problems and algorithms.



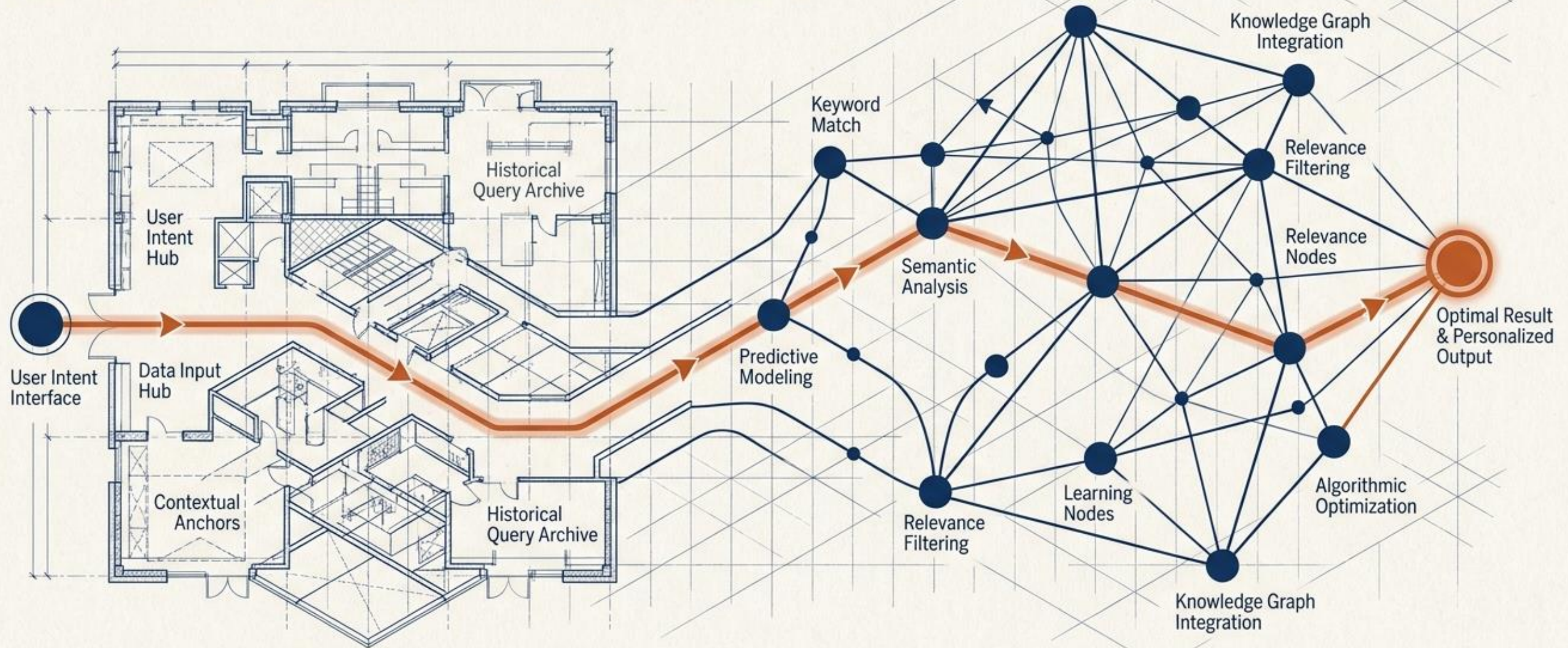
Describe graph theory, graph representations, and their real-world applications in AI.



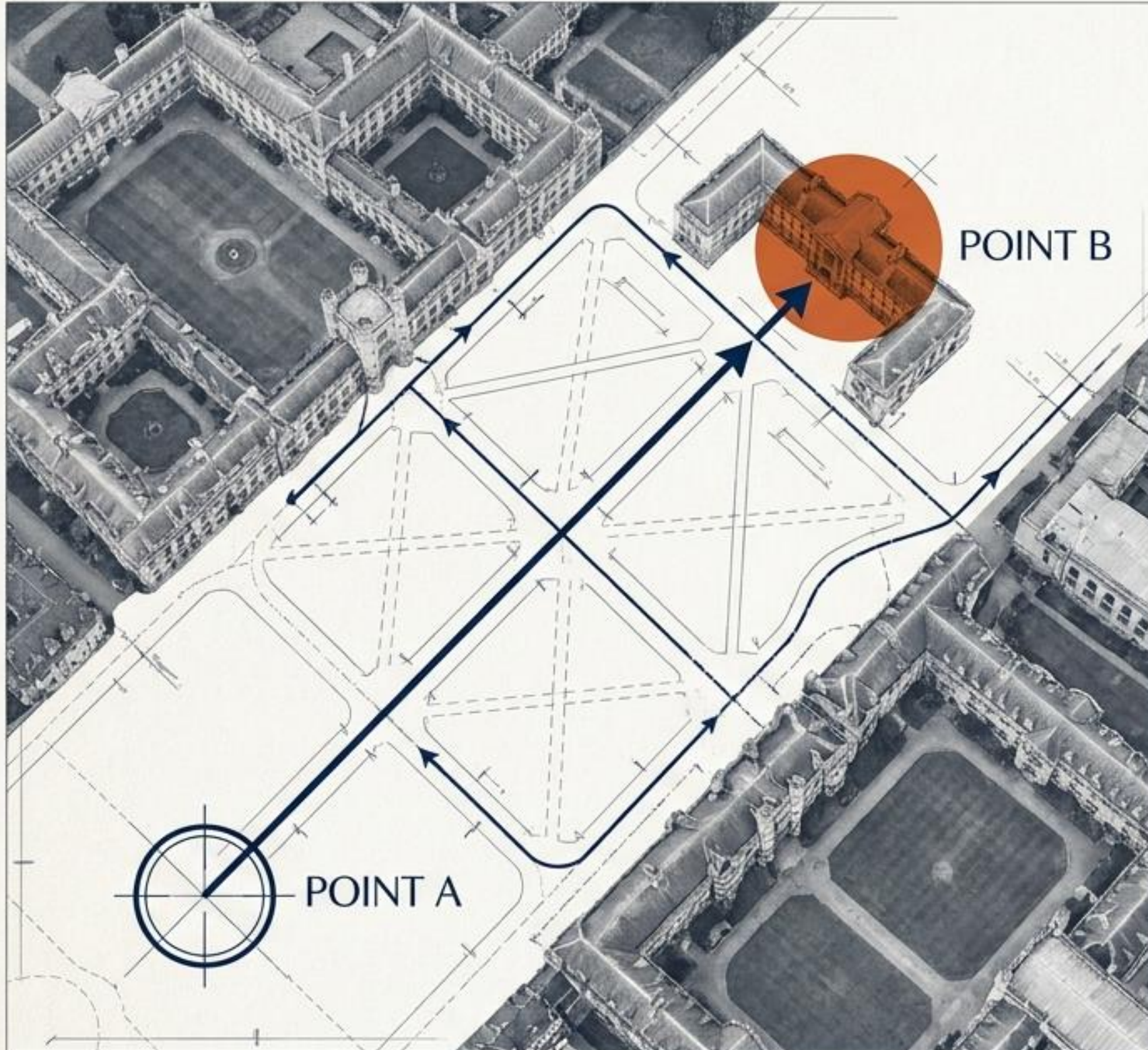
Navigating Complexity: AI Search Algorithms & Graph Theory

From initial states to optimal solutions

Algorithmic Navigation and AI Search Mechanics



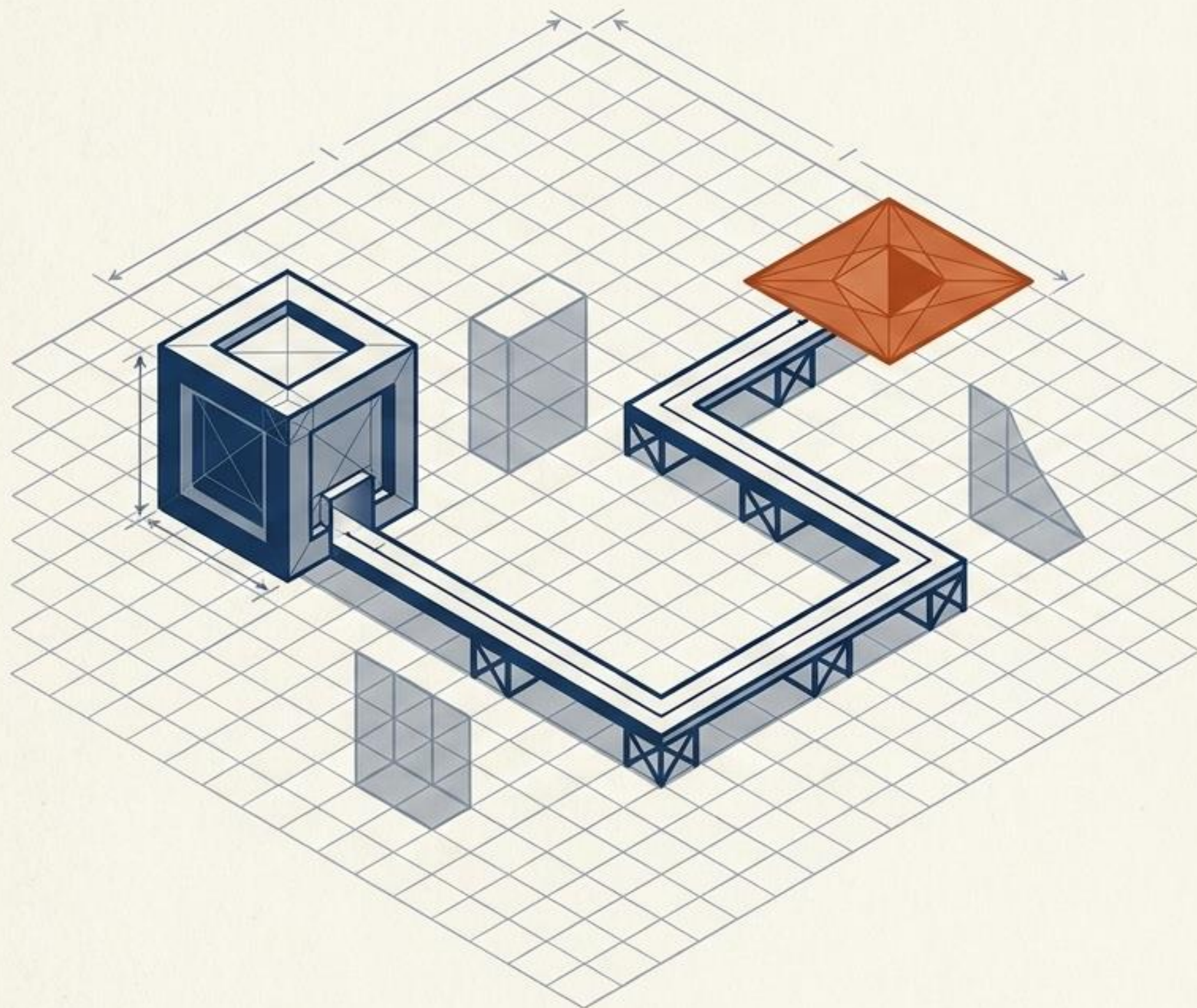
Pre- Assessment



HUMAN PROBLEM-SOLVING RELIES ON INTUITIVE PATHFINDING

Suppose you are standing in a university and you want to reach a particular classroom. What do you do?

You look at the available paths, consider where you need to go, and decide which path to take. You inherently compare the routes and choose an appropriate one.



TRANSLATING INTUITION INTO ALGORITHMIC SEARCH

AI has to make the exact same decisions as a human navigating a campus.

It starts from an **Initial State**, explores possible choices, and tries to reach a desired **Goal**.

Search: The problem of figuring out what to do when we have a specific situation that a computer is in—the environment that an agent is in.

We require that agent to somehow look for a solution to that problem across any number of different formats.

Scenario

Suppose you are playing a game of chess.

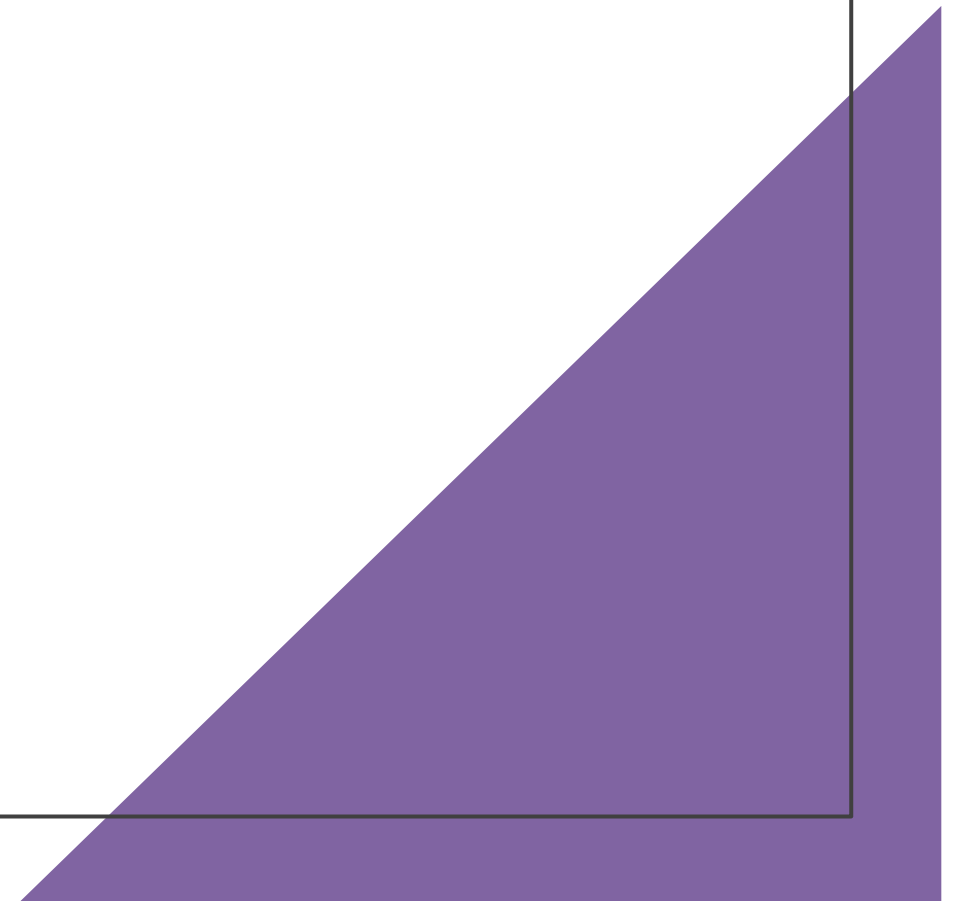
You have one current position, but you might have 20 possible moves.

After each move, your opponent may have another 20 moves.

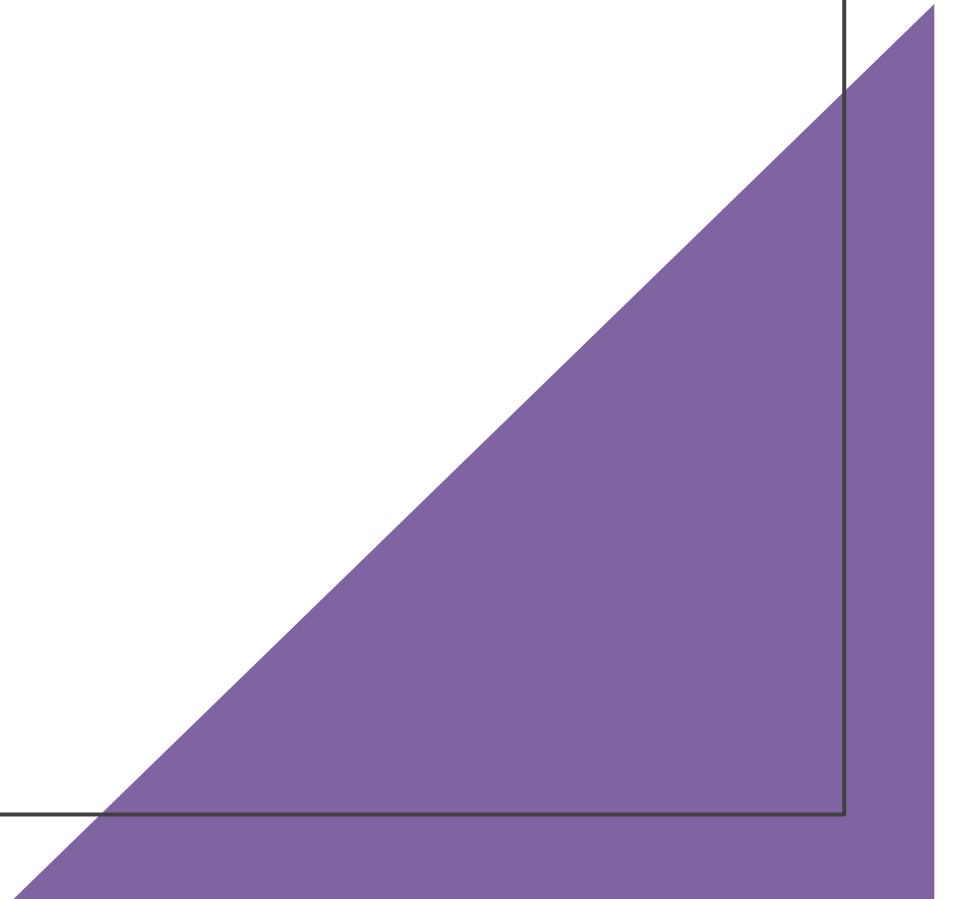
Very quickly, the number of possibilities becomes enormous.

What would happen if the computer randomly selected chess moves?

It would probably make
poor decisions

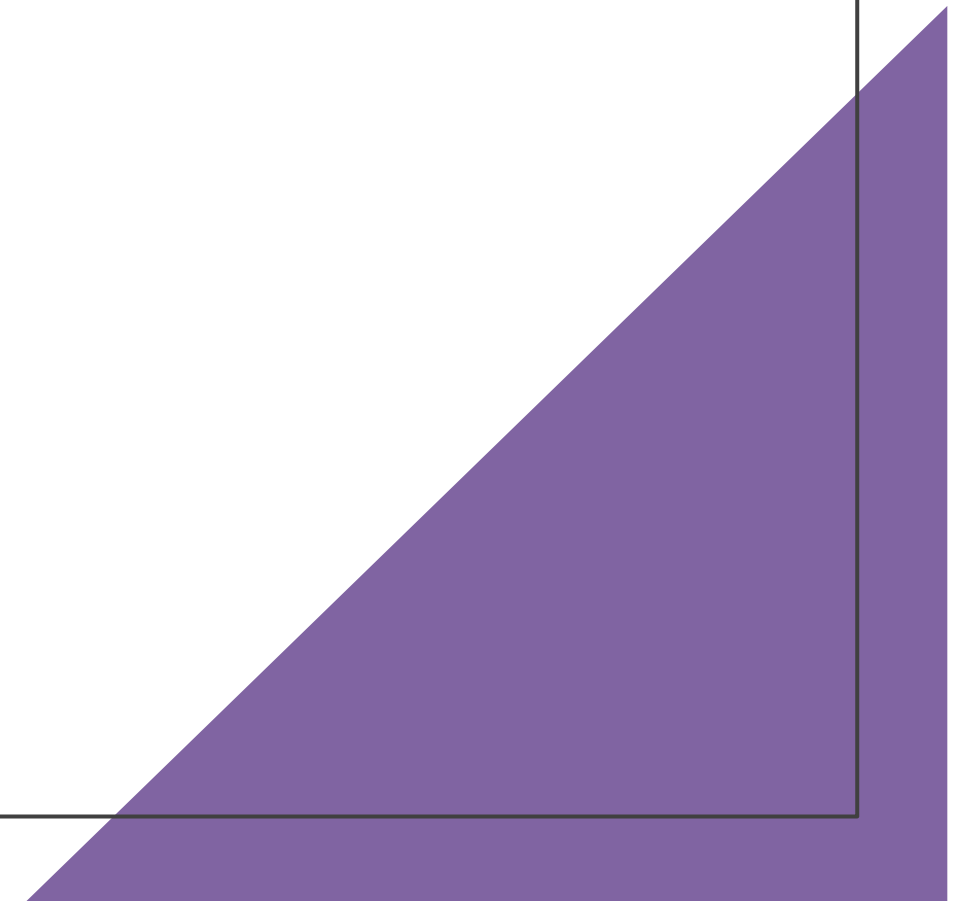


Search = Finding a solution
among possible choices/states



AI not only in chess and maze it also in:

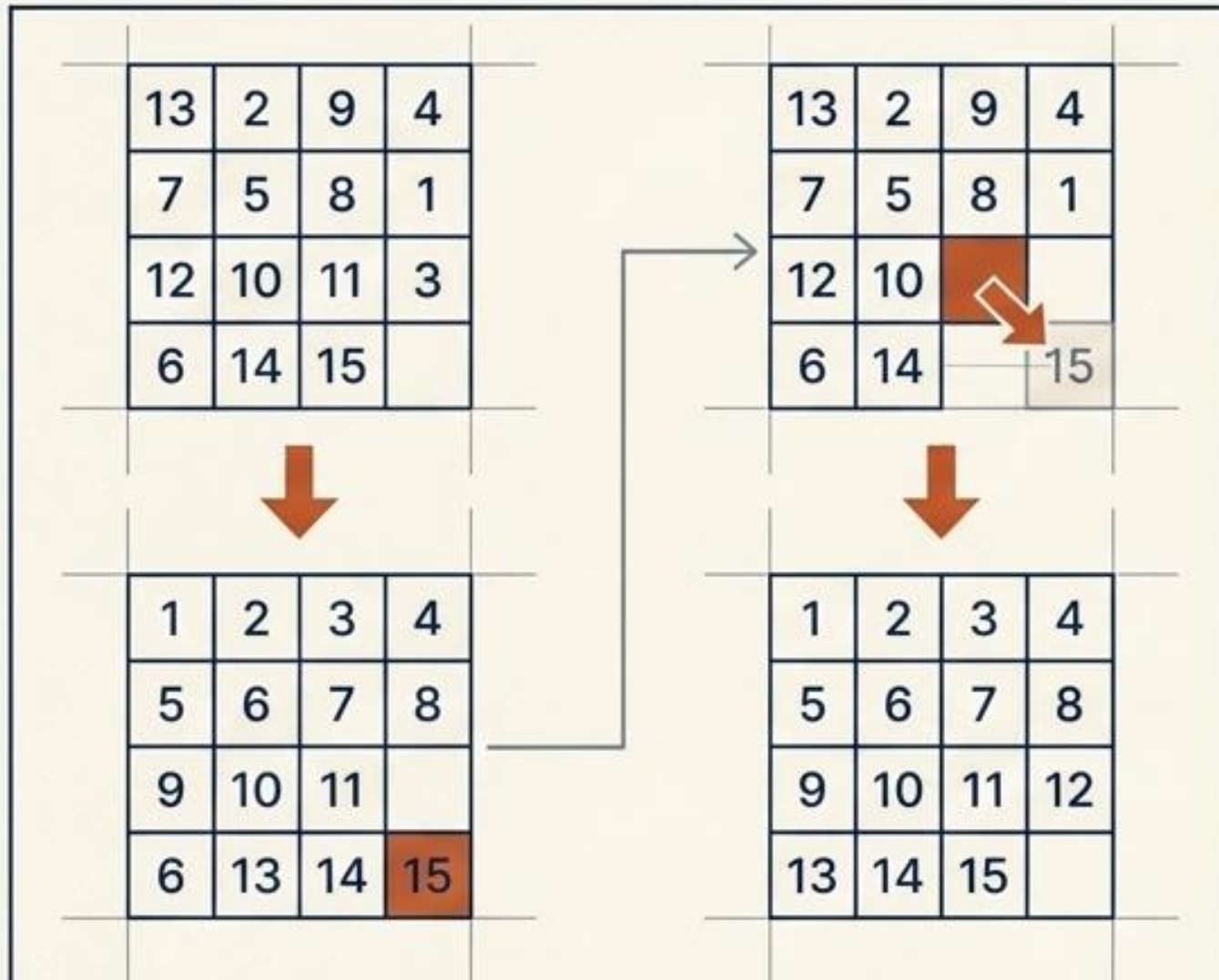
- Google Maps route finding
- GPS navigation
- Robot navigation
- Game playing
- Puzzle solving



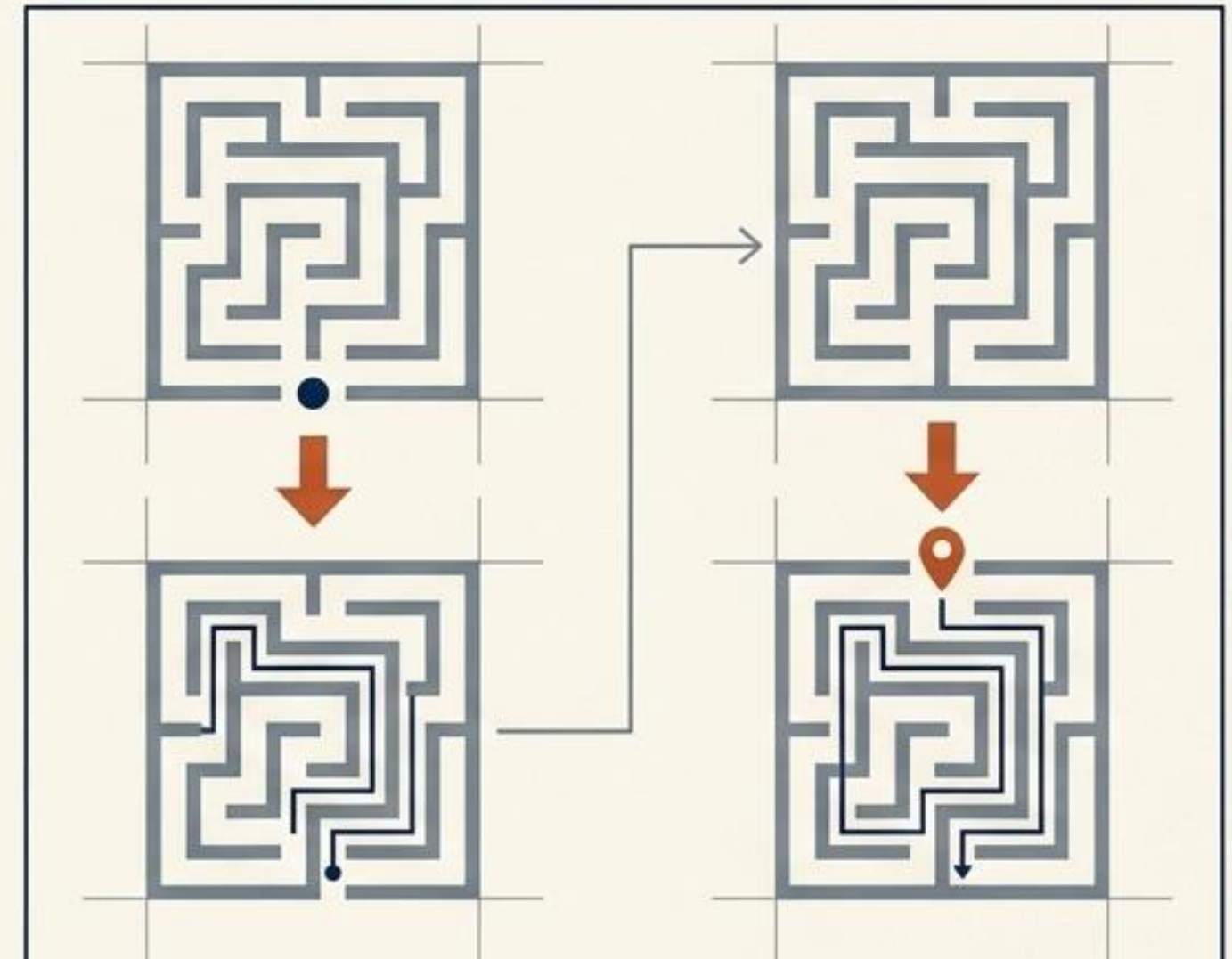
Defining the State-Space Through Classical Problems

A Solution is defined strictly as the correct sequence of actions that will take us from the initial state to the goal state.

15-Puzzle Game



Maze Game

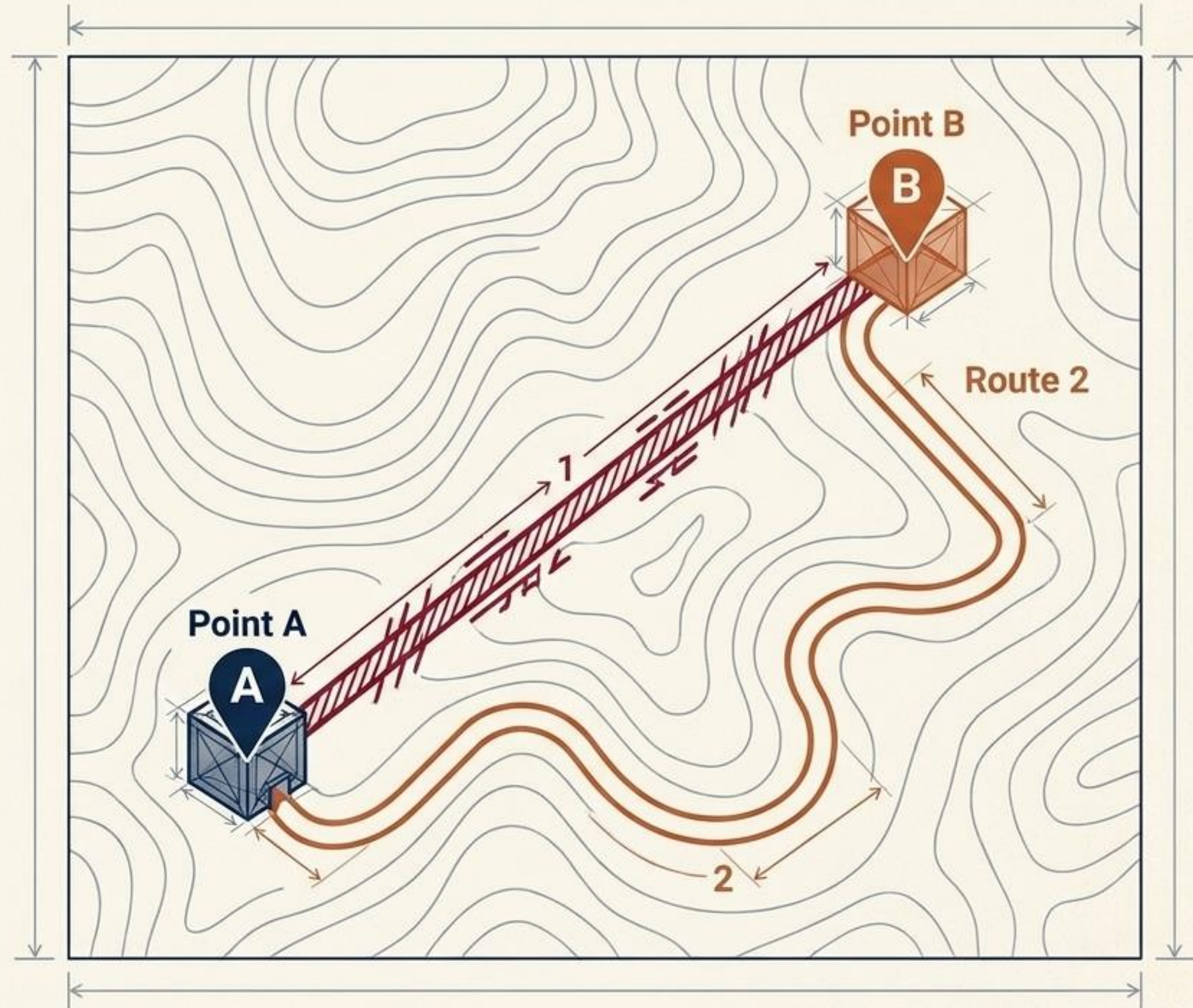


Isolating the Optimal Solution

Ideally, we would like to find not just any solution, but the Optimal Solution.

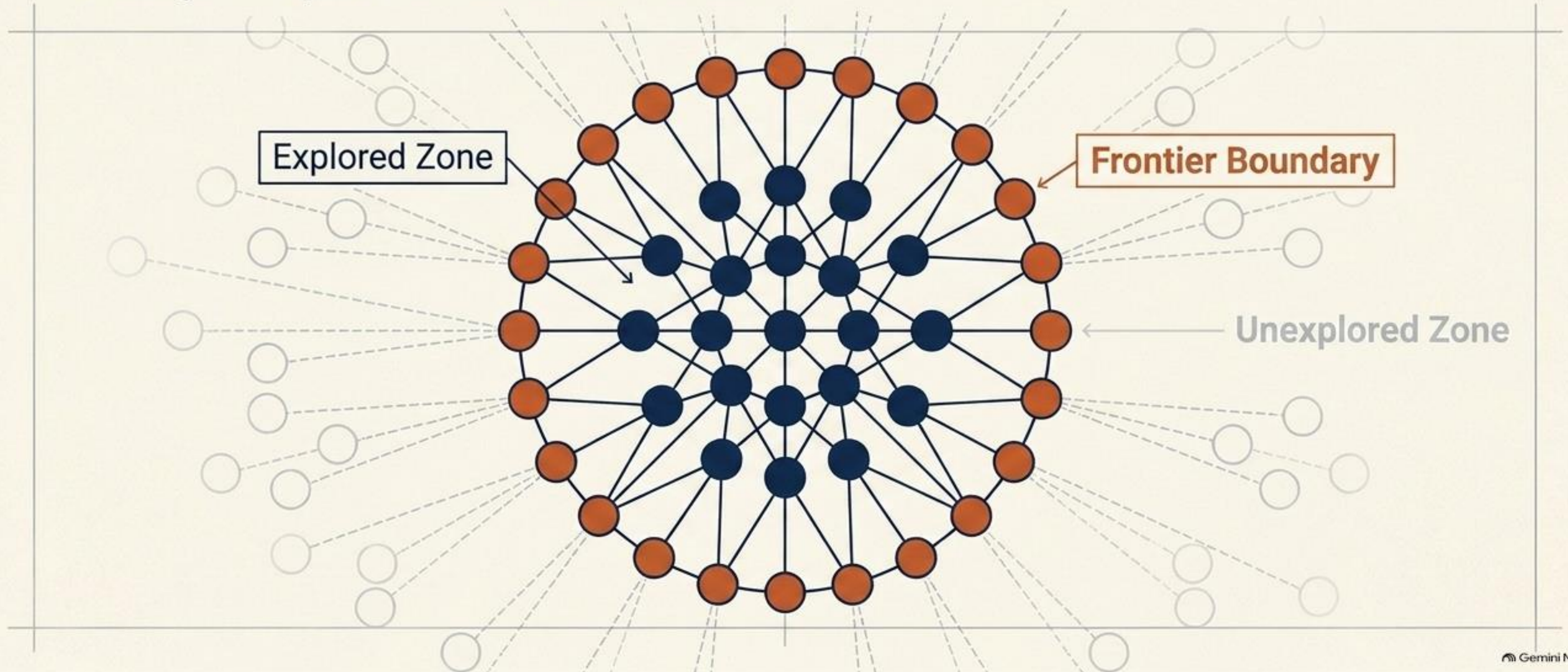
Optimal Solution: A solution that has the lowest path cost among all solutions.

Real-world application: Google Maps must figure out the best way to get from Point A to Point B, including what turns to make depending on traffic conditions.

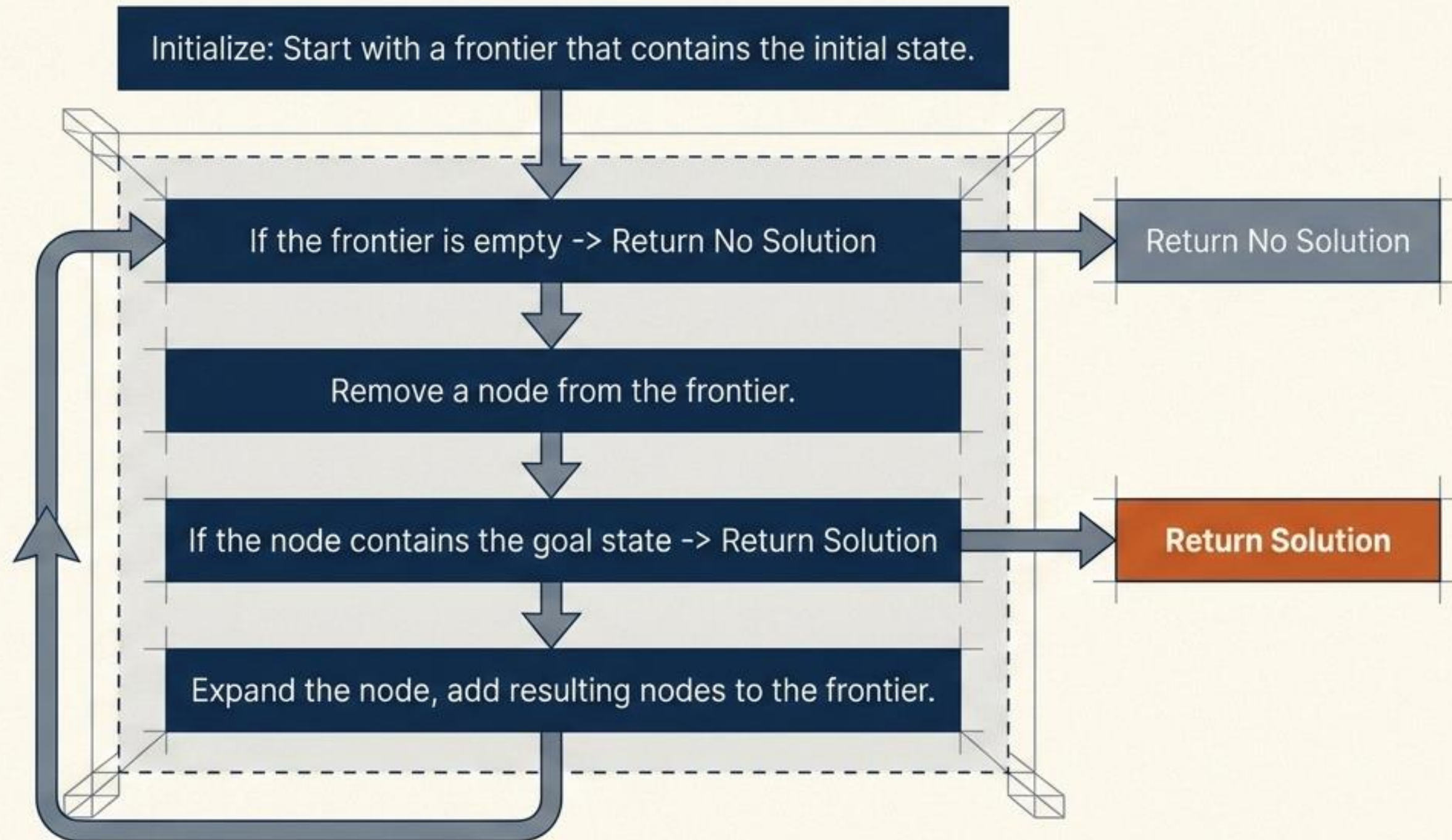


The Expansion Engine and the Search Frontier

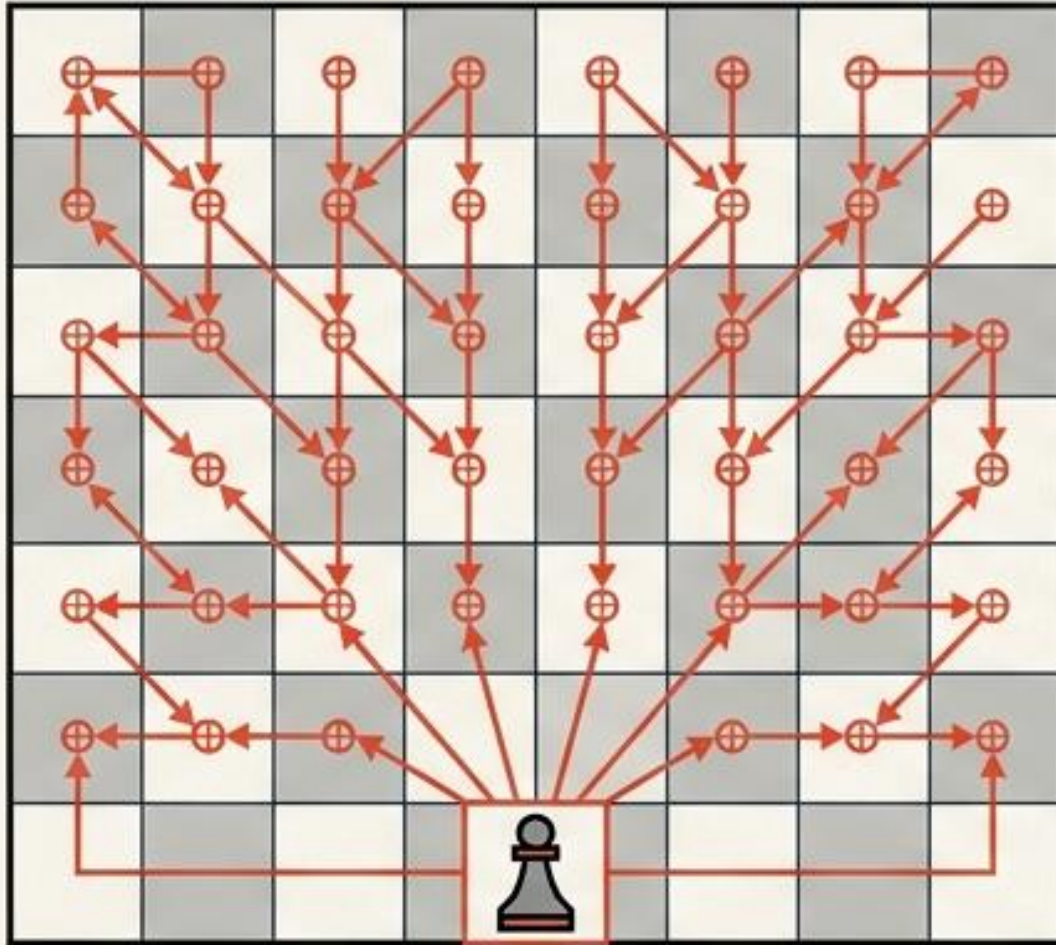
To solve search problems, the algorithm must physically expand its awareness outward. It begins precisely at the Initial State and branches outward, holding the edge of its current knowledge in a queue known as the Frontier.



The Universal Search Loop



Search is Systematic Problem-Solving



Search algorithms explore a massive set of possible solutions to find the optimal path from an initial state to a goal state.



Why are algorithms necessary? They allow AI systems to navigate complex decision spaces where the number of possibilities is enormous—whether finding the shortest route to a destination or calculating the best strategy to win a game of chess. Random choice is impossible; systematic exploration is required.

Search Problem

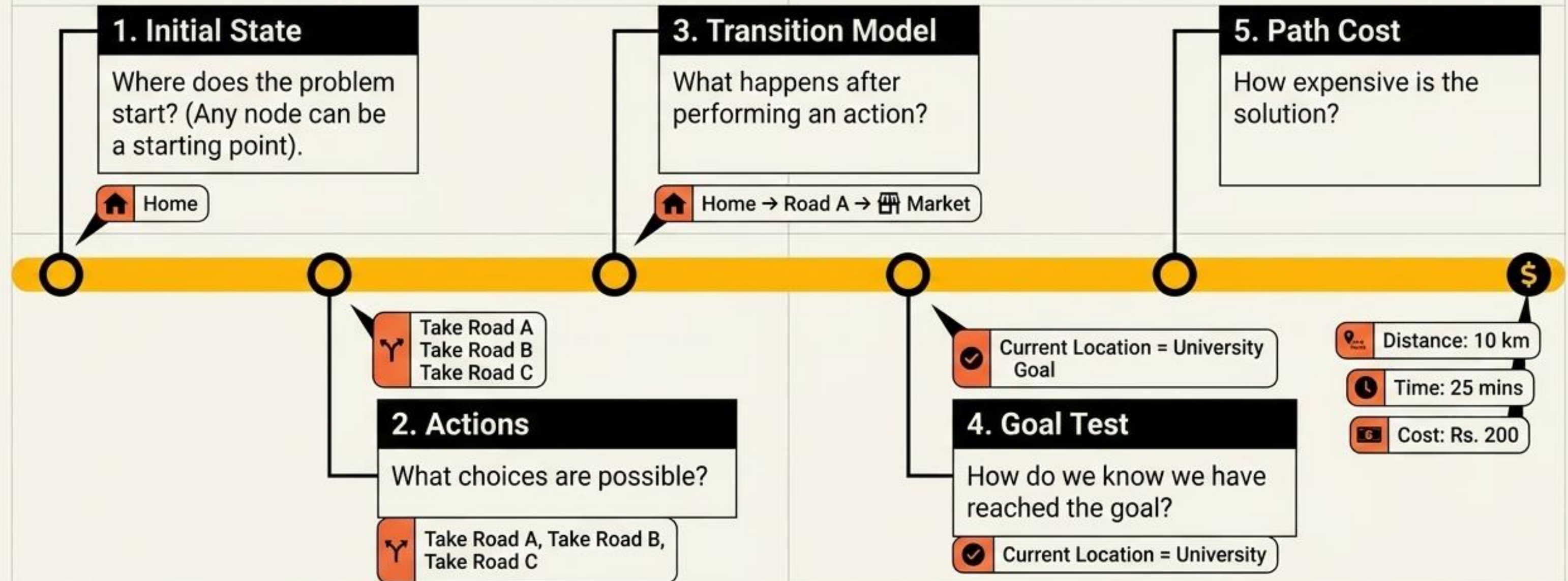
If a problem given to an AI, then it should know:

- What is the starting point?
- Where to go?
- Which actions have to perform.
- Every action create an new state
- When will get the solution.



How do we define a search problem?

The 5 Pillars of a Search Problem



The 8-Puzzle

The 8-puzzle is played on a 3×3 grid with eight numbered tiles and one empty space. The goal is to arrange the tiles in the correct order by sliding tiles into the empty space.

1) STATE

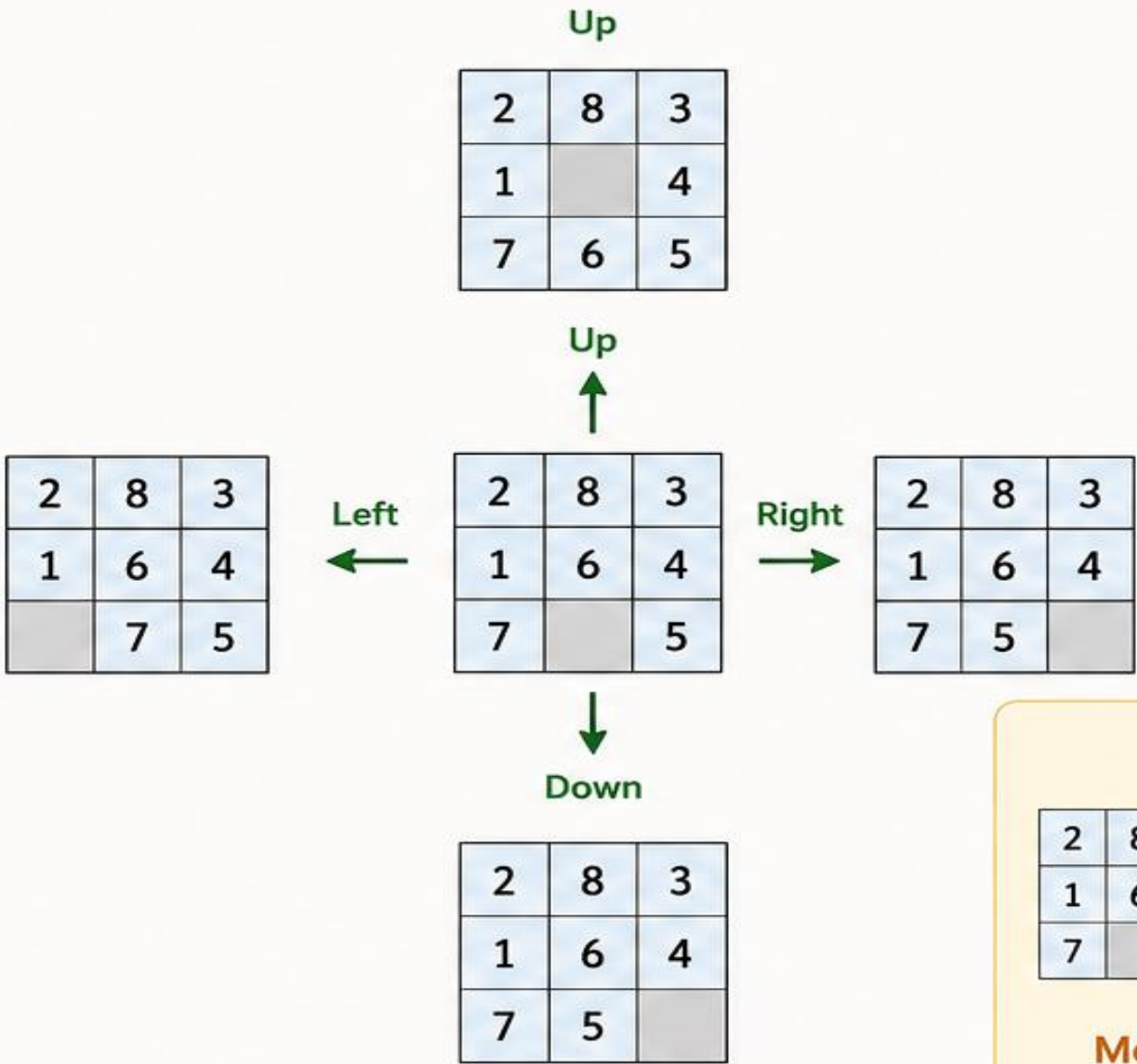
A configuration of the puzzle represents a state.

2	8	3
1	6	4
7		5

Initial State

2) ACTIONS

You can slide a tile into the empty space (Up, Down, Left, Right).



3) GOAL TEST

Check if the current state is the goal state.

Goal State (Solved)

1	2	3
4	5	6
7	8	

4) OBJECTIVE

Find the shortest sequence of moves (actions) that transforms the initial state into the goal state.

Example Solution (one possible shortest path)



Moves: Right, Down, Left, Up, Right, Down ...



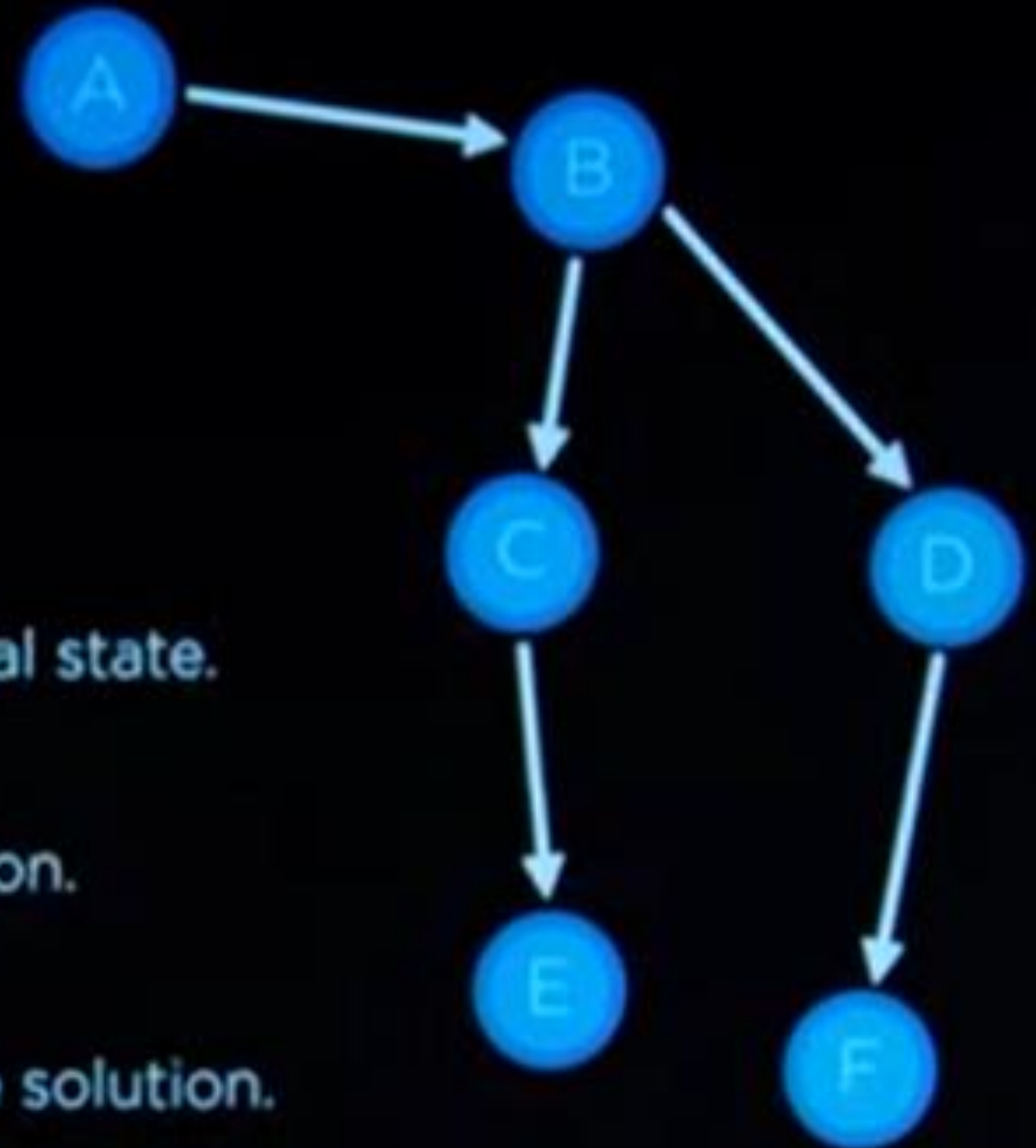
Summary

- **State:** any arrangement of the 8 tiles and the blank.
- **Actions:** slide a tile (Up/Down/Left/Right) into the blank.
- **Goal Test:** check if we reached the goal state.
- **Objective:** find the shortest sequence of actions to reach the goal.

Find a path from A to E.

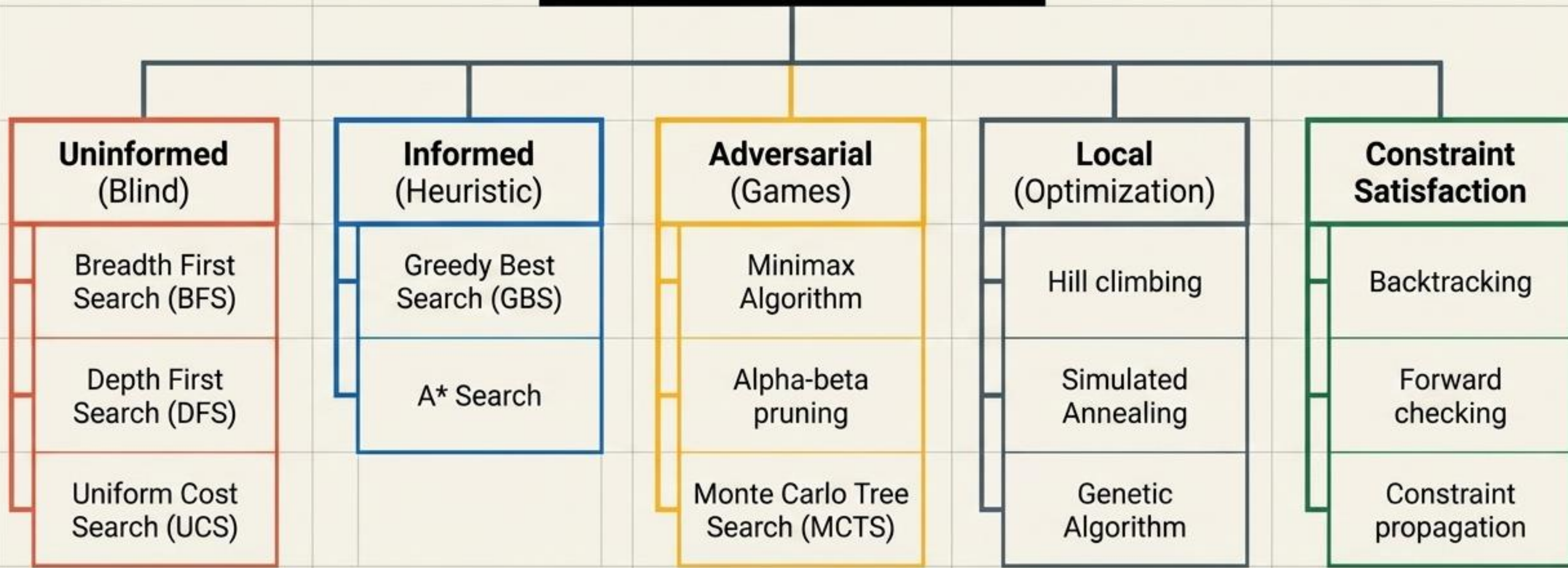
Frontier

- Start with a frontier that contains the initial state.
- Repeat:
 - If the frontier is empty, then no solution.
 - Remove a node from the frontier.
 - If node contains goal state, return the solution.
 - Expand node, add resulting nodes to the frontier.



The AI Search Algorithm Taxonomy

AI Search Algorithms



Search Paradigms Explained

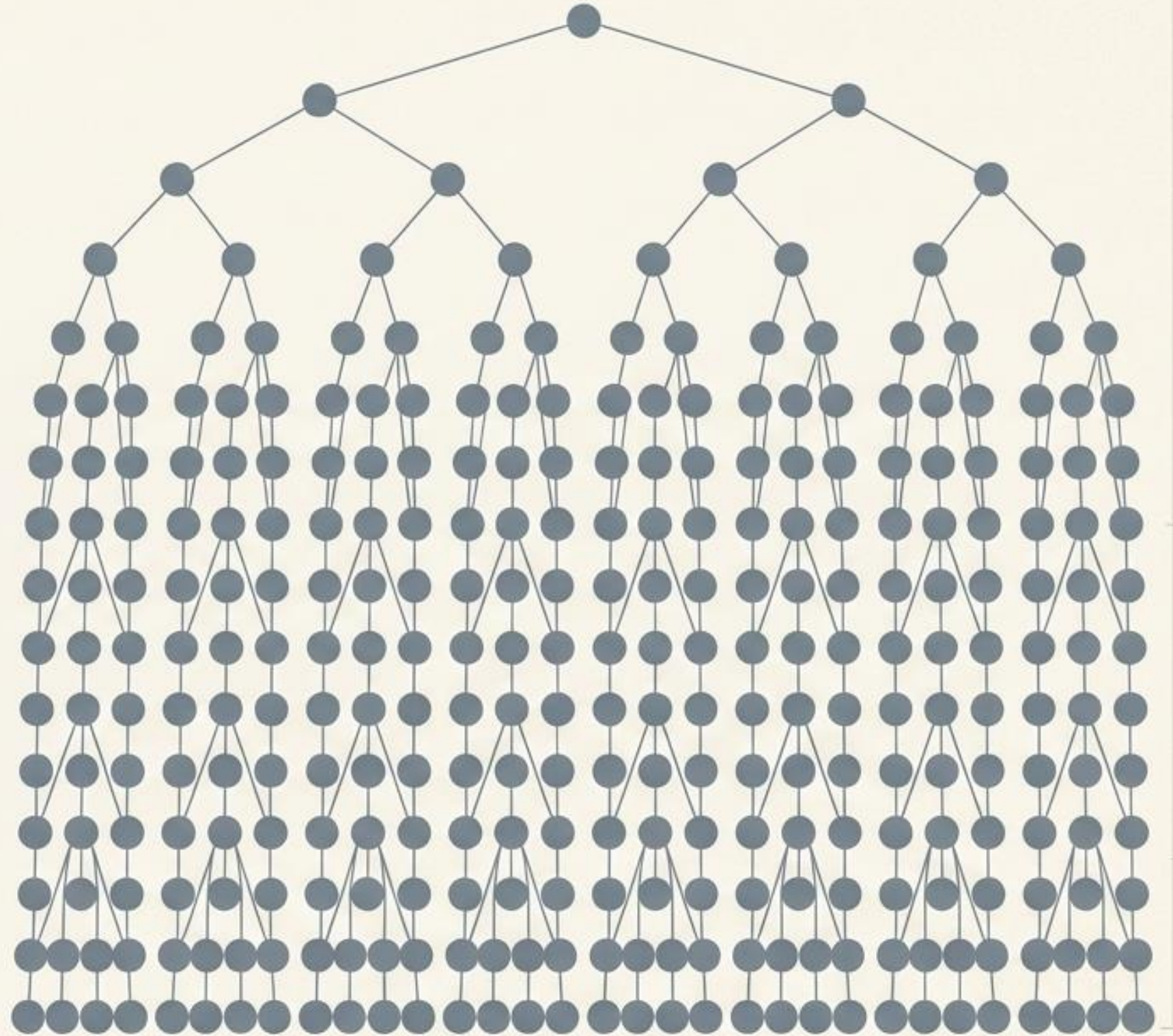
	Uninformed Search	Operates with zero additional information about the goal. Relies on pure, systematic exploration.
	Informed Search	Uses problem-specific knowledge (heuristics) to actively guide and accelerate the search toward the goal.
	Adversarial Algorithms	Designed for competitive scenarios (like games). Must consider and counter the opponent's moves to find the optimal solution.
	Local Search	Operates on a single candidate solution. Used exclusively for optimization problems rather than standard pathfinding.
	Constraint Satisfaction	Focuses on feasibility. Solves problems by satisfying strict constraints over variables rather than finding a path.

Diagnostic Matrix: Uninformed vs. Informed Search

Dimension	Uninformed Search	Informed Search
Strategy	Brute Force / Blind	Heuristic / Guided
Knowledge Usage	No domain knowledge	Uses active information
Processing Speed	Time Consuming (Slow)	Quick Solution (Fast)
System Complexity	High (Time & Space)	Less Complexity
Optimality	Guaranteed Optimal Solution	Compromised Optimality
Application	Maze Game	Google Maps

Paradigm 1: Uninformed Search Mechanics

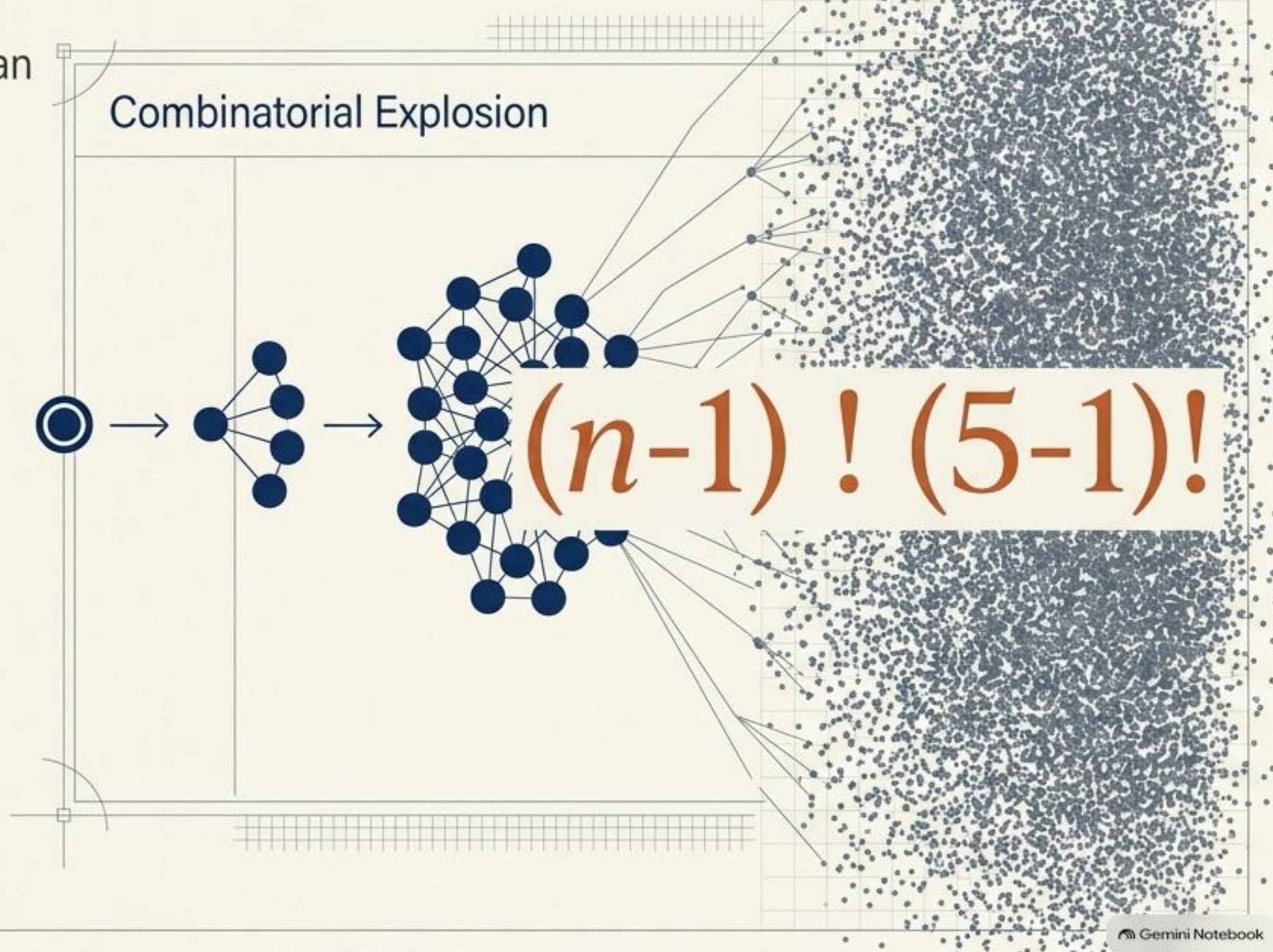
- Also known as brute force or blind searching.
- Explores absolutely all states in an environment; after exploring all states, it eventually reaches the goal state.
- Executes the search without any external information or guidance—there is zero knowledge about the domain.
- Real-world analogy: Solving a Maze Game through sheer exhaustion of options.

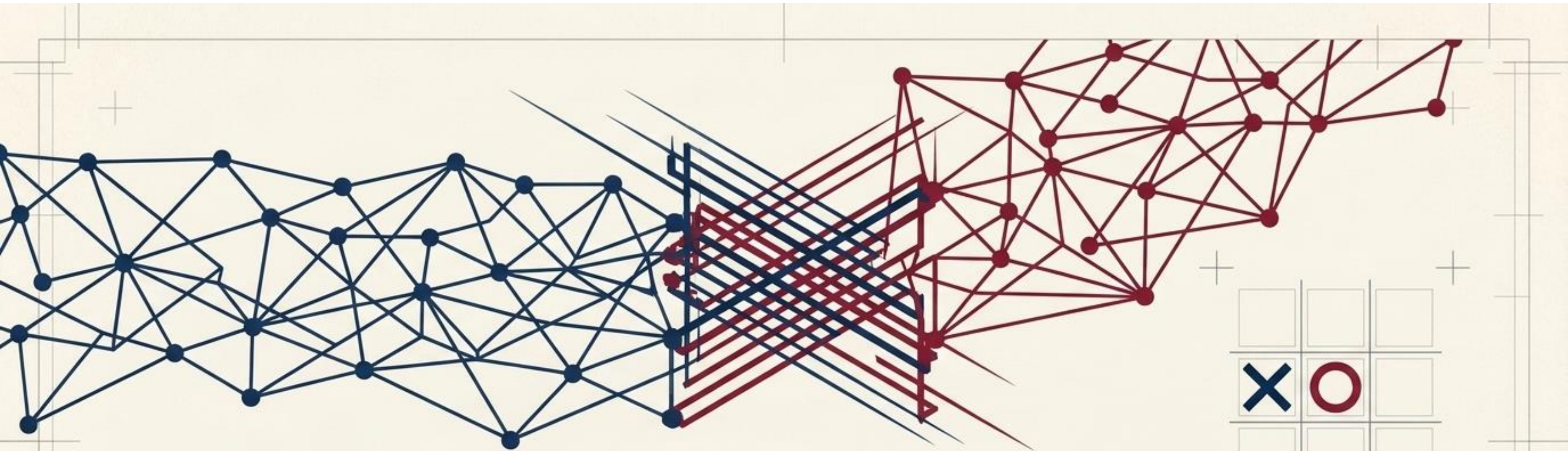


The Computational Weight of Brute Force

While Uninformed Search guarantees an Optimal Solution by checking every possibility, it suffers from severe limitations at scale:

- Highly Time Consuming
- Massive Complexity: Exponential demands on both processing time and memory space.
- The mathematics of factorial growth ensure that searching without guidance quickly exhausts computational limits.





Paradigm 3: Adversarial Search

Competition Games: Search algorithms must adapt when the environment is no longer passive.

- In Adversarial Search, another intelligent agent is actively working against the AI's attempt to reach its goal state.
- The search loop must factor in the anticipated actions of an opponent.
- Real-world example: **Tic-Tac-Toe**

UNINFORMED SEARCH (BLIND SEARCH)

Uninformed search algorithms explore the search space **without** any domain knowledge or heuristic. They use only the problem definition (initial state, actions, goal test, and path cost).

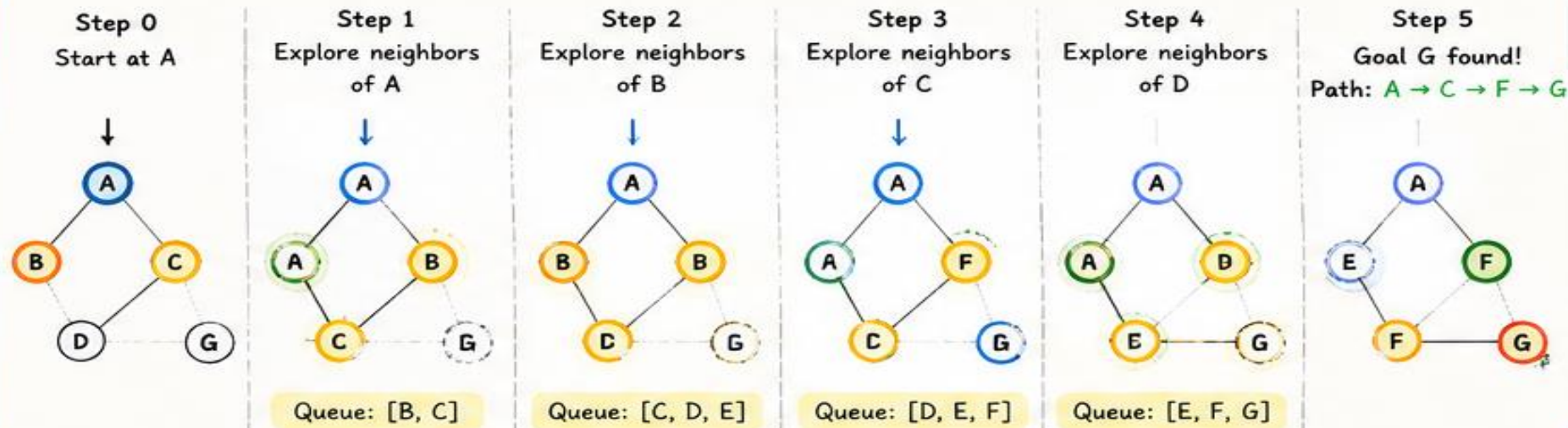
★ Key Characteristics

- ✓ No heuristic or domain knowledge
- ✓ Uses only problem definition
- ✓ Explores nodes systematically
- ✓ Guarantees completeness (for BFS, DFS with finite depth limit, UCS, IDS)
- ✓ Optimality depends on the algorithm
- ✓ Time and memory can be high

Common Uninformed Search Algorithms

- Breadth-First Search (BFS)
- Depth-First Search (DFS)
- Uniform Cost Search (UCS)
- Iterative Deepening Search (IDS)

How BFS (Uninformed Search) Works



✓ Advantages

- Simple to understand and implement
- Guaranteed to find a solution if one exists (complete)
- Finds the shortest path in unweighted graphs (BFS)
- Optimal for equal-cost paths (BFS)

✗ Disadvantages

- Can be very slow for large search spaces
- Requires a lot of memory
- Inefficient if the solution is deep in the tree (DFS)

Real World Example

Example: Finding the shortest route in a city using BFS

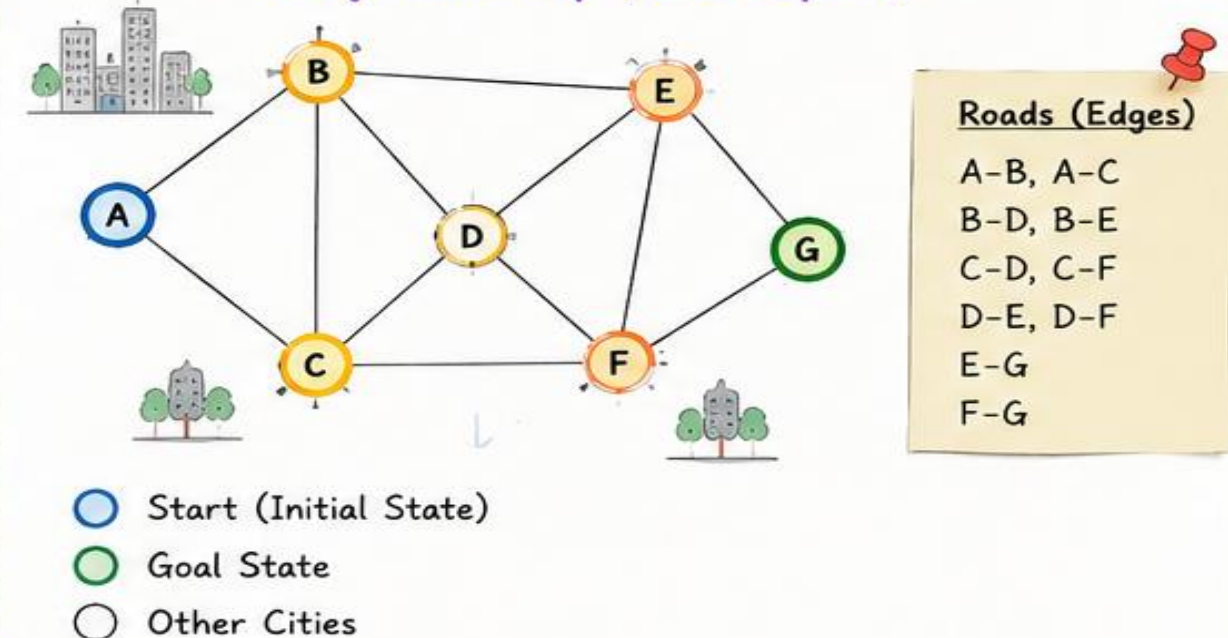
Problem: You are in City A and want to reach City G. You know the possible roads (connections) between cities, but you have no idea which path is shorter. BFS explores all nearby cities level by level until it finds the goal.

⚙️ Technical Details

- **State Space:** Cities as nodes, roads as edges
- **Initial State:** City A
- **Goal State:** City G
- **Actions:** Move from one city to an adjacent city
- **Cost:** Each road has equal cost (e.g., 1)
- **Algorithm Used:** Breadth-First Search (BFS)
- **Data Structure:** Queue (FIFO)
- **Time Complexity:** $O(V + E)$
- **Space Complexity:** $O(V)$

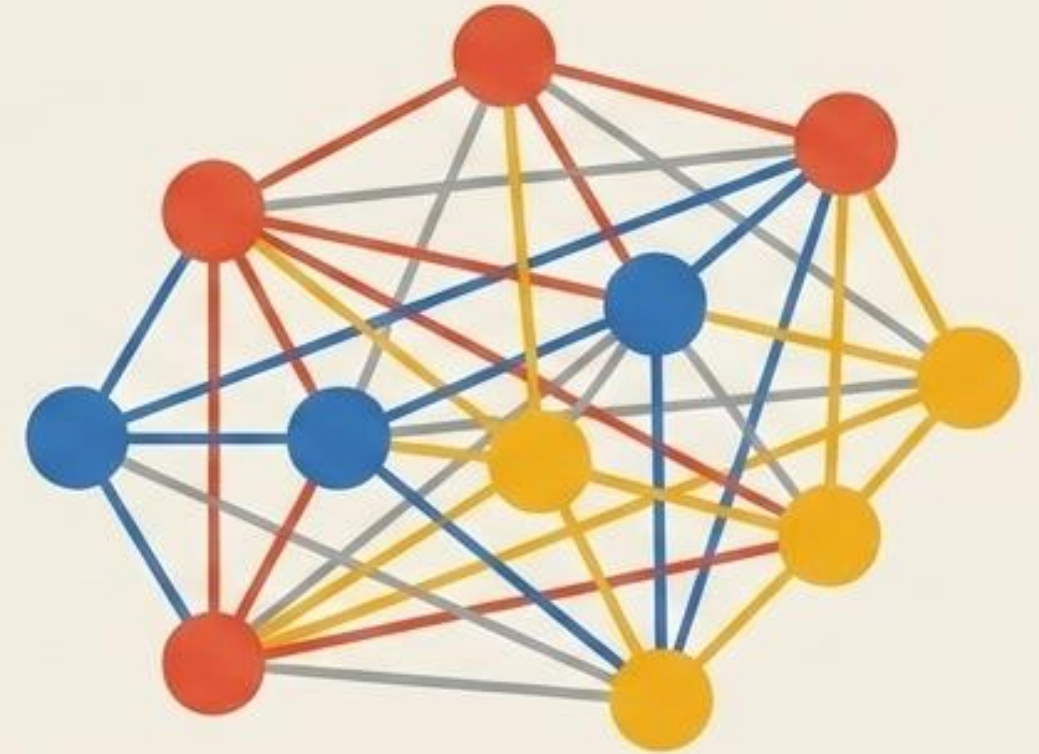
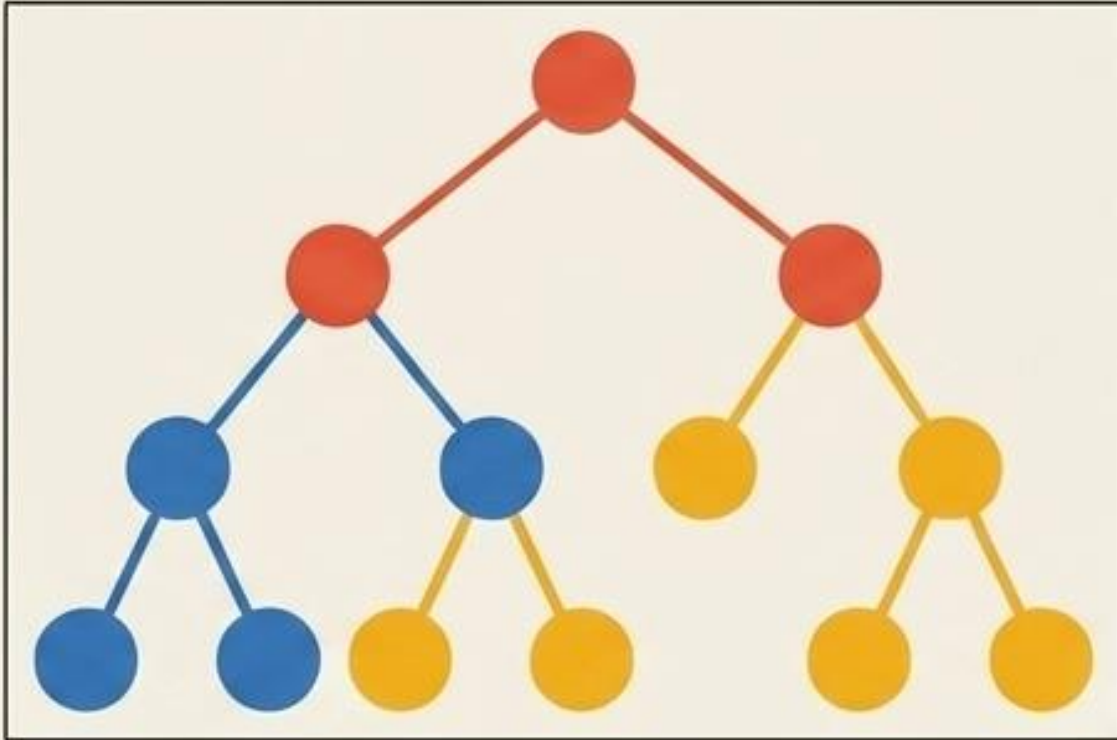
V = number of cities (vertices), E = number of roads (edges)

City Road Map (State Space)



But what structure will we use to represent the problem?

The Terrain: Graph Theory



Core Definition:

A graph is a vital data structure comprised of a collection of network nodes. It looks like Google Maps—different places connected by different routes or paths.

The Components:

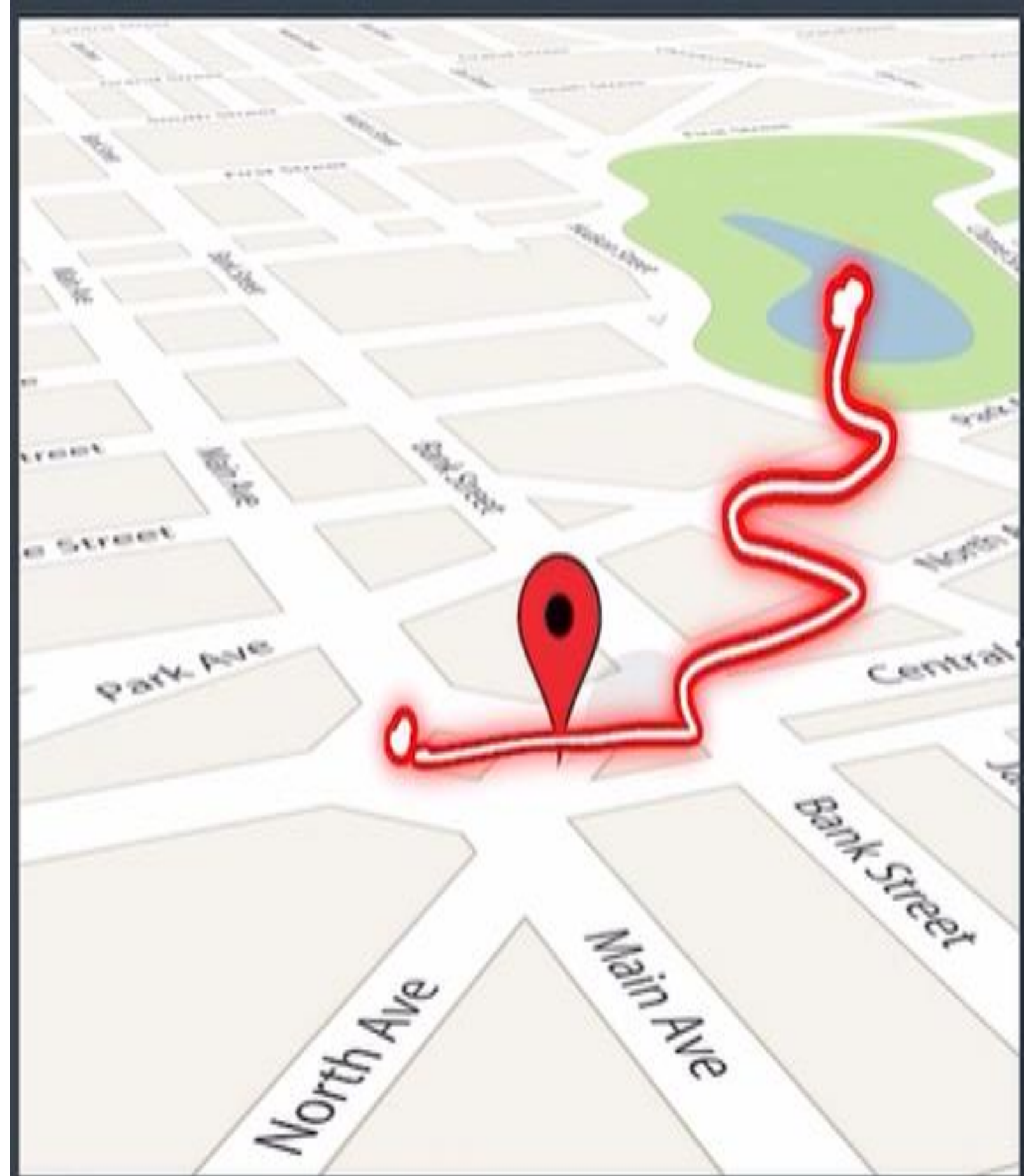
Vertices: The nodes (think of them as corners).

Edges: The lines connecting the vertices.

Tree ka gar level up kr dia jaye tau graph hai.

Google Maps.

Cities or locations can be represented as nodes,
while roads can be represented as edges



In a university campus map, what could be a vertex?

A building.

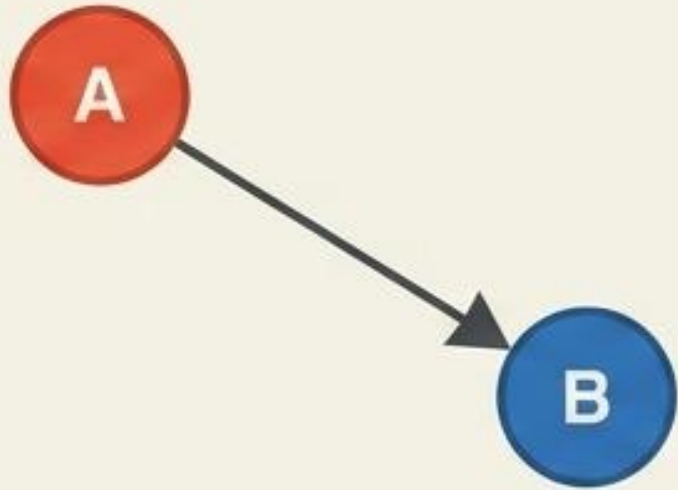
And what could be an edge?

A road/path connecting buildings.

- **Transition**

But not all edges behave in the same way. Some allow movement in one direction, some in both directions, and some have costs.”

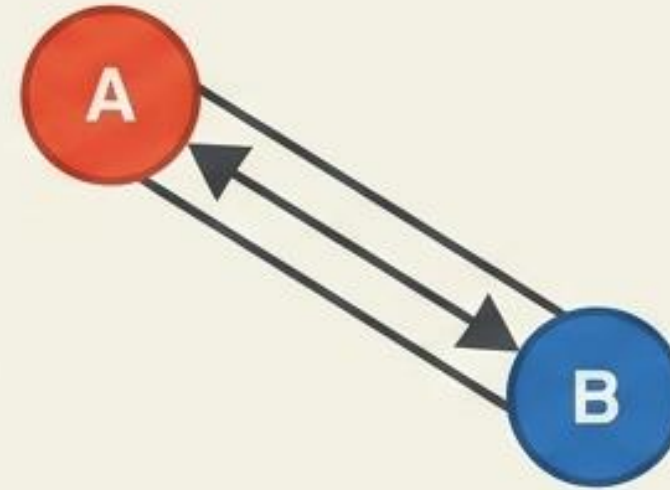
Edge Mechanics & Traversal



Unidirectional Edge

Can travel from A to B, but not B to A.

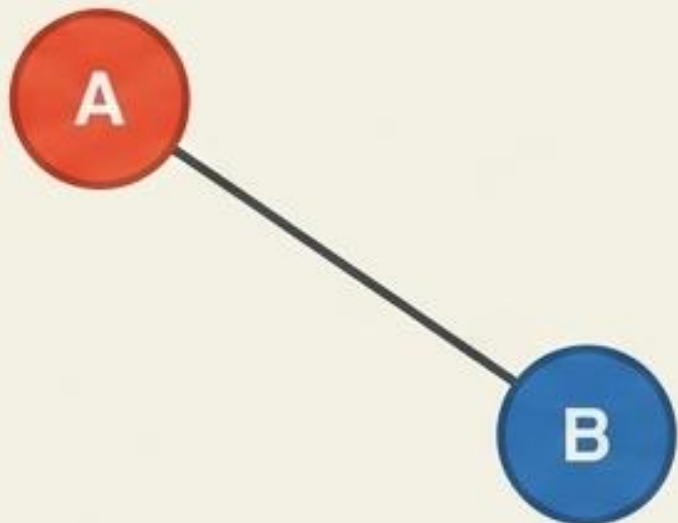
A	A	One way Road
B	B	



Bi-directional Edge

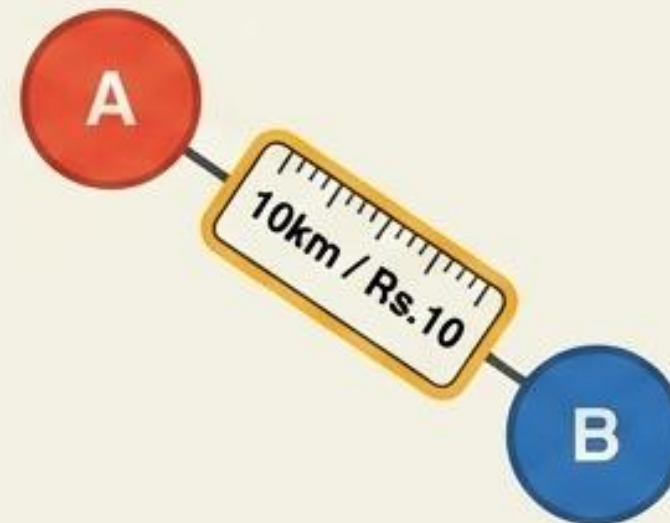
Two-way travel.

A	A
B	B



Undirected

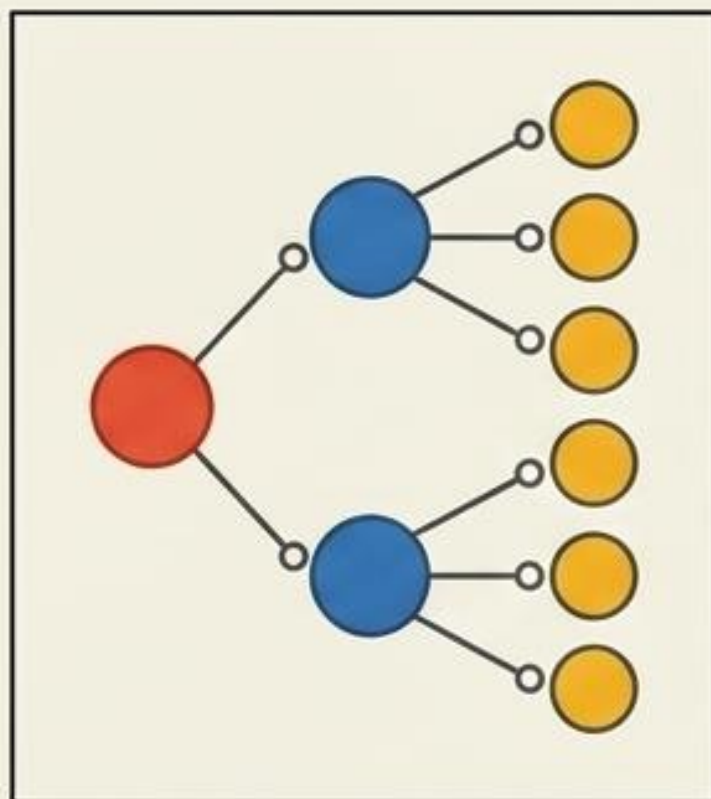
Connections where specific direction is simply not defined.



Weighted Edges

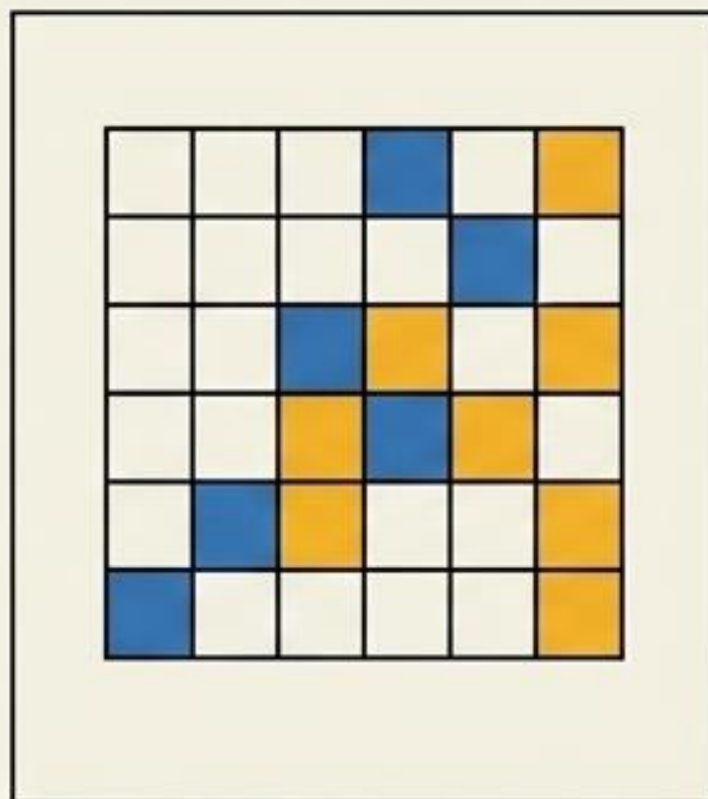
Connections associated with a specific value. Values can be visualized as distance (e.g., 10km), cost (e.g., 10 Rupees), or time. Weights can be positive, negative, or 0.

Storing the Map: Graph Representations



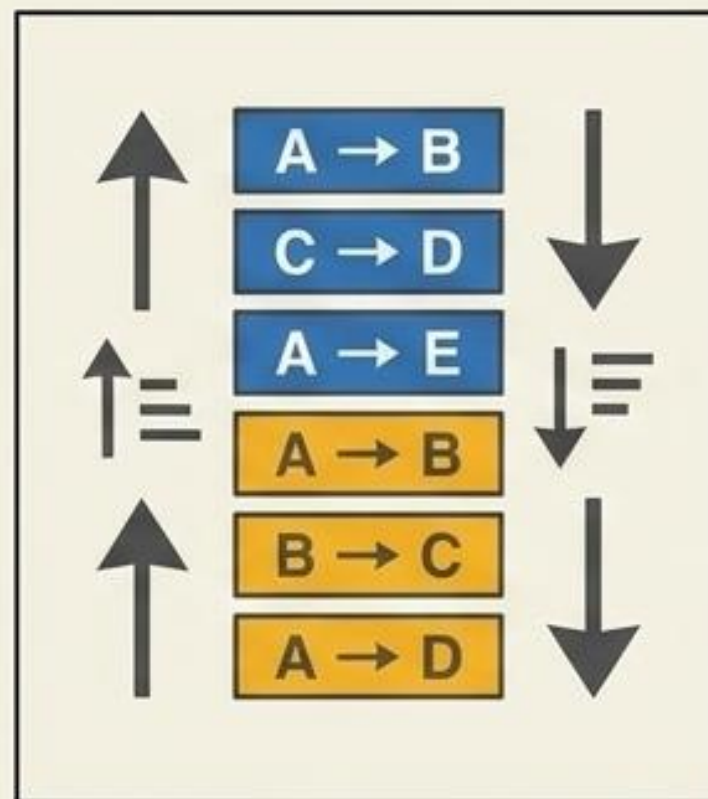
Adjacency List

A hierarchical structure mapping node-to-neighbor lists.



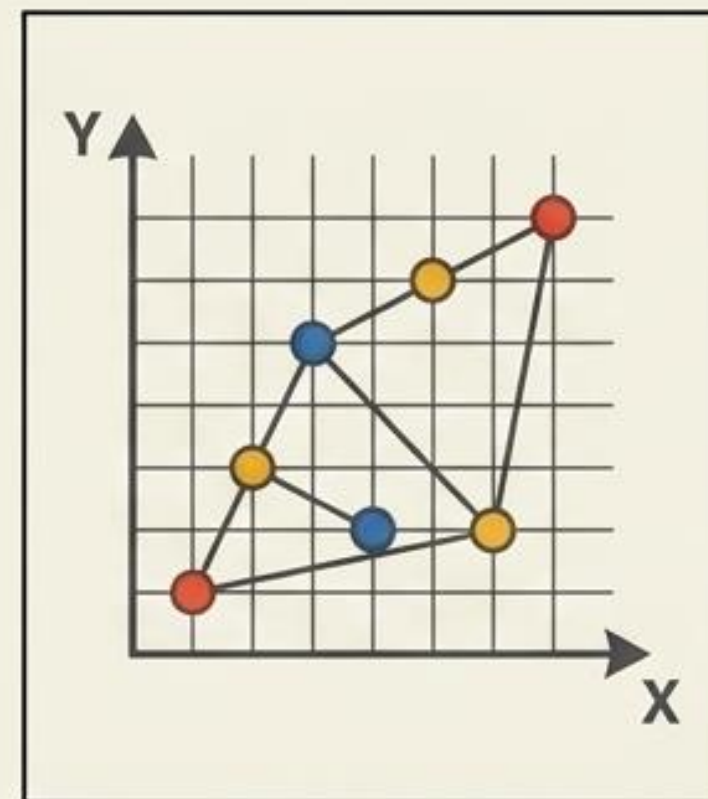
Adjacency Matrix

A 2D grid structure mapping every possible edge intersection.



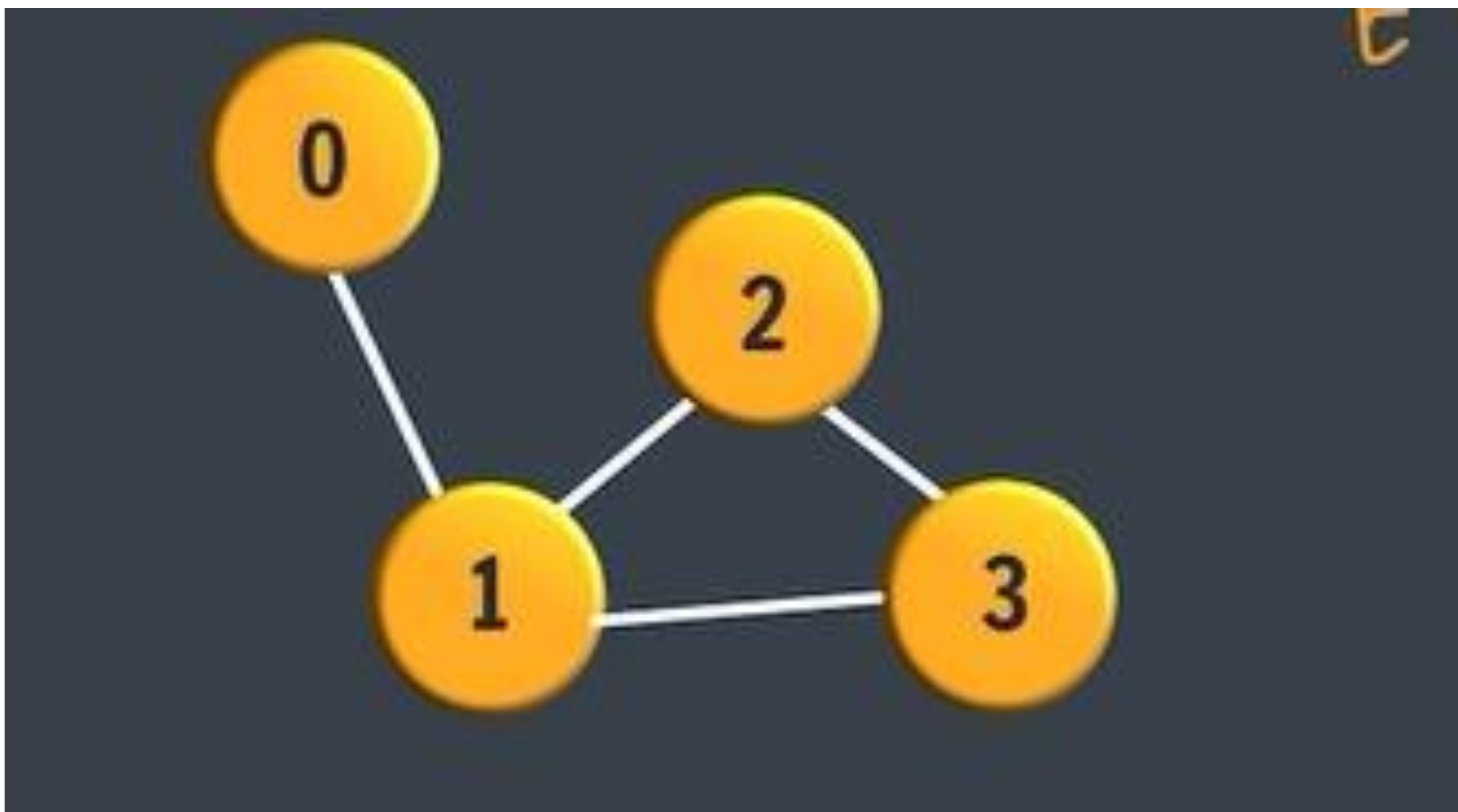
Edge List

A simple, linear list of connections optimized for easy sorting.



2D Matrix

An implicit graph structure.



Structural Diagnostic: List vs. Matrix

Adjacency List (List of Lists)

Logic: Every vertex stores only its neighbor information. Optimized for space and time. Does not store extra info.

Example (4 Vertices, 4 Edges, Undirected):	
List store for 0 :	[1] 
List store for 1 :	[0 , 2 , 3] 
List store for 2 :	[1 , 3] 
List store for 3 :	[1 , 2] 

Adjacency Matrix (Grid)

Logic: Stores binary information of all possible edges. Requires massive space even if unconnected.

	0	1	2	3
0	0	1	0	0
1	1	0	1	1
2	0	1	0	1
3	0	1	1	0

Example: From 0 to 0 = no edge (0).
From 0 to 1 = edge exists (1).

From 0 to 0 = 1 = no edge (0).
From 0 to 1 = edge exists (1).

Time Complexity: $O(k)$

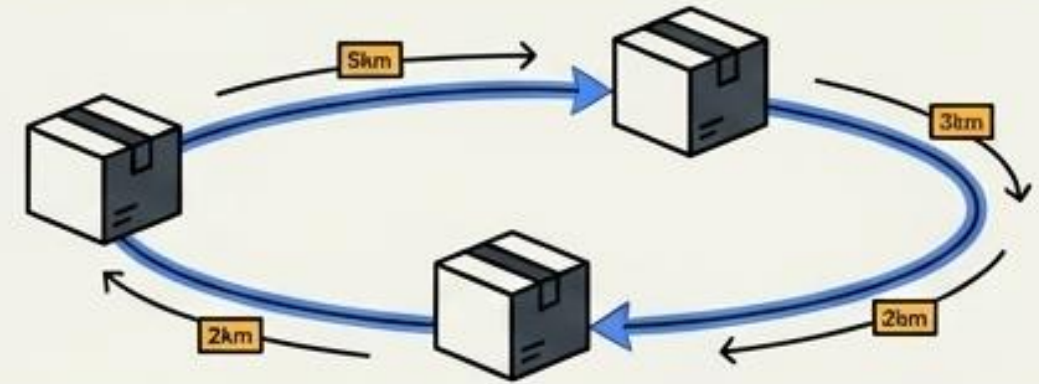
Space and Time Complexity: $O(V^2)$

Graph Applications: Logistics & Networks



Navigation & Maps

Calculating the shortest paths. Used heavily in cab booking systems and flight time calculations.



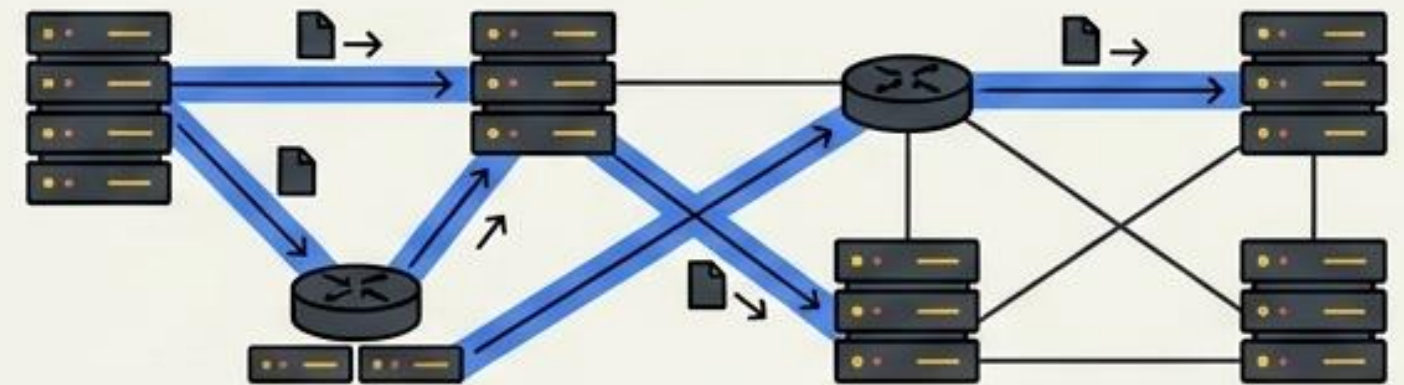
Logistics

Optimizing delivery networks by finding the shortest cycle routes for fleets.



Social Infrastructure

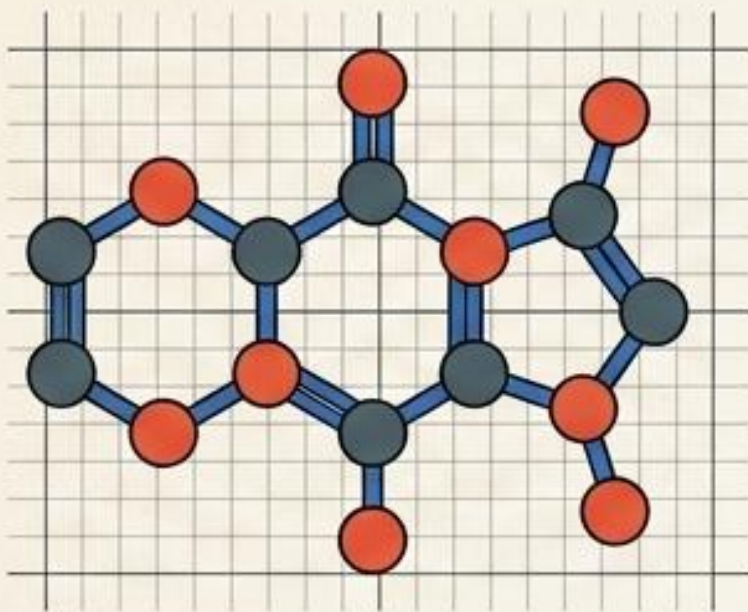
Underpinning the complex web of connections in social networks.



Internet Routing

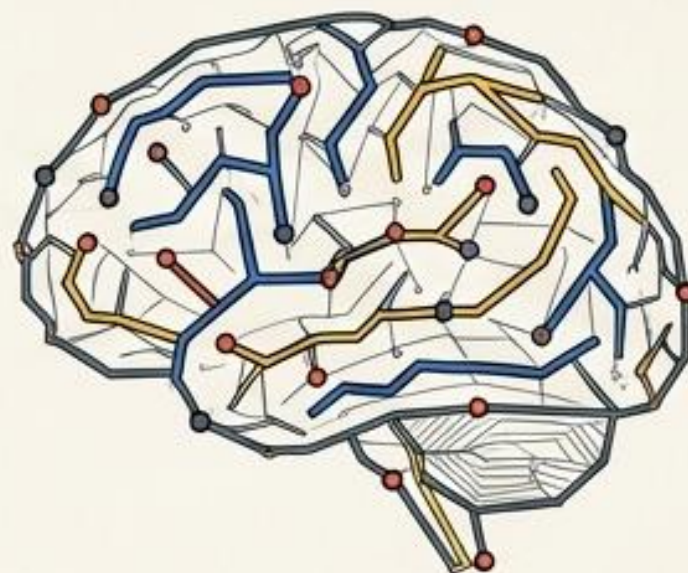
The internet is a collection of networks. Graph routing algorithms dictate how routers send data packets from one place to another.

Graph Applications: Advanced Science & Computing



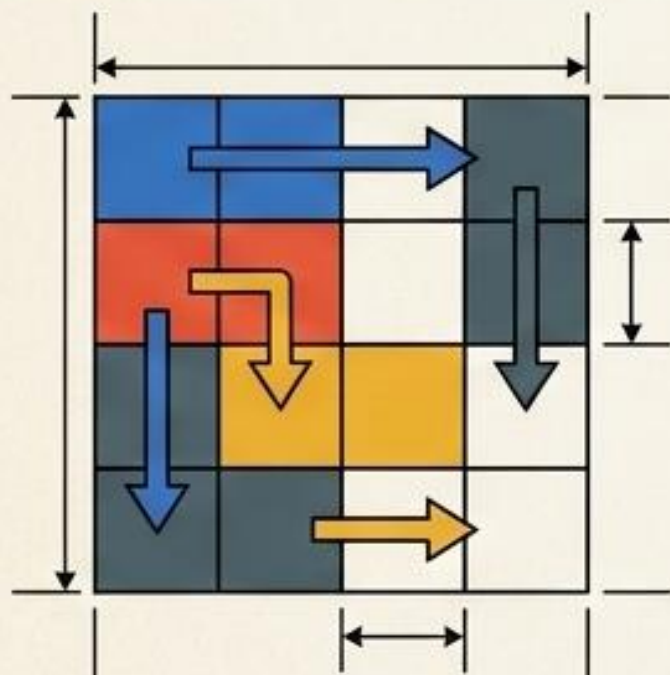
Physics & Chemistry

Visualizing complex molecule structures via computer programs.



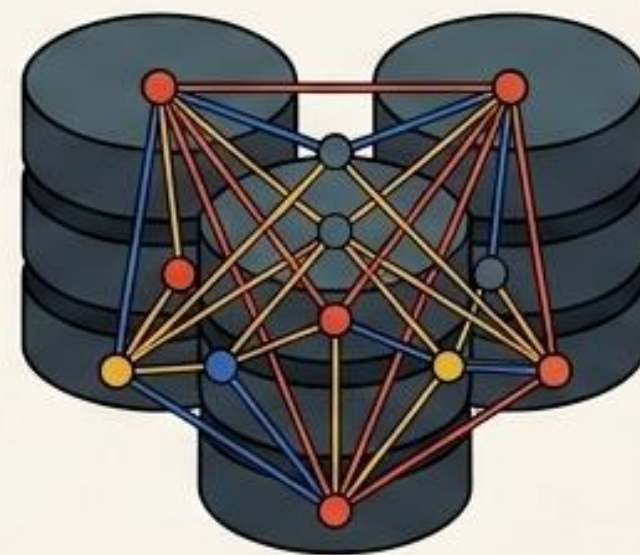
Biology & Psychology

Human brain mapping software; modeling neurons as individual nodes and biological connections as edges.



AI & Computer Vision

Machine learning computation graphs. In Computer Vision, used for image segmentation and fluid fill algorithms (identifying empty corners/places to fill).



Graph Databases

Systems like Nebula and Neo4j. Essential for heavy-duty fraud detection and recommendation systems.

Synthesis: The Architecture of Intelligence



The Peak

The Solutions (Applications):

Routing packets, fraud detection, chess AI, fluid fill.
The real-world impact of optimized decision spaces.

The Middle

The Navigators (Search Algorithms):

BFS, A*, Minimax, Local Search.
The strategies used to systematically traverse the terrain.

The Base

The Terrain (Data Structures):

Graphs, Adjacency Lists, Matrices.
You cannot search without a map.

To solve a complex decision space, you must first construct the mathematical terrain (Graph), then deploy the right explorer (Search).